How to read this record

Date: 2026-10-16

Status: This directory is the official statement.

`TBdecompile/record/` is the consolidated record of the ToolBook II 8.1 work. The loose sheets under `tbworkingpapers/tbwpNNNN/` are the trail those findings were written on. They are not a second current record. When a loose sheet and this directory disagree, this directory wins. Chapter 18 names every loose sheet and the section that absorbed it.

The dated folders `trail/20261012/`, `trail/20261014/`, and `trail/20261015/` stay as the notes of those days. `trail/20261016/00-consolidated.md` is the short correction this volume expands. Read those folders for the trail. Do not read `trail/20261015/00-how-toolbook-operates.md` section 2, or `trail/20261015/01-theories-of-operation.md` section 1, or `trail/20261014/02-still-open.md` opening paragraph, as the runtime create path.

Grades

Grade	Meaning
Known	Seen in a binary, a live trace, or a matching export. A later observation would have to contradict the evidence, not just a reading of a listing.
Thought	Fits the evidence and is the working model. Not yet watched end to end.
Conjecture	Plausible. Not tested.
Not true	Was a working idea. Later evidence closed it. Do not search it again.
Superseded	An older note still says this. The later note wins. The older path is listed so it is not reopened by accident.

Order

File	Subject
01-corpus.md	What the books are, and which machines can run them
02-formats.md	`.tbk`, `.sbk`, `.tbx`, `.fbk`
03-runtime.md	Modules, launch seam, NIDA system layer
04-objects.md	Live object, file open, the unnamed reader
05-graphics.md	Painters and what is stored versus computed
06-openscript.md	Bytecode, handlers, the modern subset
07-debugging.md	SoftICE, WDW, Unicorn, Bochs, where the cipher lives
08-naming.md	What has been named, and what a name is worth
09-modern.md	Foolbook, Foolcourse, Foolversity, TBpipeline
10-open.md	Ranked gaps, and the notes that are closed
11-keepers.md	One-boot live addresses and decompile line numbers
12-wdw-spine.md	The book-open spine and what the three traces hit
13-specimen-pages.md	P1040, P430, P440 inside that record, P420, P1170, and the TBX draw inventory
14-openscript-bytes.md	Encoding, the back-reference fix, corpus counts
15-system-books.md	`.sbk` tiers, NIDA shell, type-name table
16-cipher.md	TBCBT segment 7, the keyed path, the empty-key path, and the author write that does not load TBCBT
17-modules.md	Module roles, including the ones that are not the reader
18-absorption.md	Every loose sheet, and where its claim went
19-heap-stack-resmgr.md	NearScript heap, the value stack, `RESMGR*`, and the callers the breakpass did not name
20-cdb-obj.md	`CDB*` is the book API. `OBJ*` is the object file and object memory. Not a header and its definitions.
21-stress.md	Stress of the Python models. Not a timing of `TB80BAS`.

File	Subject
22-value-vbx-execscript.md	The value bus, a VBX control, and `EXECSCRIPT`. Three mechanisms. None of them is the `0x76` page object.

Lookup tables that are too long to typeset live in `maps/`:

File	What it is
maps/rtm-ne-to-ghidra.md	TB80RTM NE segment to Ghidra space
maps/bas-ne-to-ghidra.md	TB80BAS NE segment to Ghidra space
maps/wdw-to-ghidra.md	One boot's WDW selector:offset against the decompile
maps/bochs-20261020.md	One boot's Bochs selectors. Dead after the next reboot.

NE segment numbers are not Ghidra `CodeN` names. Segment 1 is `1000`, segment 2 is `1008`, and the usual step is 8, with gaps. Live selectors change every boot. Offsets inside a segment do not.

What this record will not hold

Binary dumps, SoftICE console logs, and personal correspondence stay out of this repository. Names of correspondents are masked. The correspondent already used in the public notes is JohnDoe1. `who-is-who` is not part of this record.

Which older sentence to trust

If a note written on or before 2026-10-15 puts `CDBSENDCREATE` between file open and the first painted object, that sentence is superseded. The correction is chapter 4. Instructor calls ordinal 51 when it creates or rewrites an object. A lesson open does not.

If `tbworkingpapers/tbwp0001/openscript_notes.md` says the keyword table decodes to "italic" or "systime", that sentence is superseded. The bug was copying bytes `0xC0`–`0xFF` from the output stream. Chapter 14 is the encoding that actually decodes the handlers.

Absence from a Ghidra HTML export is not absence from the binary. The listings have undisassembled runs. A candidate is closed only when the disassembly or a live trace shows what the function actually does.

If a note says `HEAP*` or `TBHEAP*` allocates the `0x76`-byte object, or that `TBHEAPCLEARCRITSECT` has no definition, that sentence is superseded. Chapter 19.

The printable form of this directory is TBdecompile-v0.0.11.pdf. That file is the whole book. The same book is also the separate sheets in manual/20261027/: front matter, the contents, and one file per appendix. A sheet that would reach 100 pages is broken between headings. `python3 manual/build.py` reprints them.

1. The corpus

Claims

Id	Grade	Statement	Source
C-001	Known	The lesson files in this work are Nida Corporation electronics lessons, written as Click2learn ToolBook II 8.1 shells. The book version word is `01 00 00 08`. This Nida is not the drug-abuse institute that shares the letters.	TBWorldview WV-001

Id	Grade	Statement	Source
C-002	Known	The catalog is a Navy ATT menu export (`nida:m3search`): menu `0_MASTER3` plus 20 rating tracks. 611 `.tbk` files are on disk. 290 catalog ids are unique. 321 files are not in the catalog.	courses listing, project notes 2026-10-08
C-003	Known	The worked specimen is Logic Families, `9e954d01.tbk`. It is not self-contained. It names the NIDA system books and jumps into `nida81.tbk`.	lesson header at file `0xB1A96`
C-004	Known	Windows 98 can step the engine. 32-bit Windows XP can run it. 64-bit Windows cannot run the 16-bit NE images. Windows 11 is outside the range.	WV-028
C-005	Known	The engine calls 16-bit `USER`, `GDI`, and `KERNEL`. A startup log that reports `USER32`, `GDI32`, and `KERNEL32` missing is not a broken install.	WV-027
C-006	Not true	ToolBook 8.1, as held here, is a 32-bit product that needs Windows 10 or earlier. The images in hand are 16-bit NE. ToolBook 9.0 is the 32-bit product.	`TBdecompile/trail/20261014/03-tbx.md`

What the set is

The books are computer-aided instruction for Navy electronics ratings. The menu export lists tracks such as GSE, EM, IC, GM, FC, ET, STG, AE, ATI, AO, ATO, AS, TM, CTM, FT, ETSS, STS, CTT, MN, and MV, at versions V03.2 through V03.4. `courses.xml` is that menu. It is not a drug-abuse curriculum.

A lesson is a ToolBook II 8.1 book plus a shell. The shell is `nida81.tbk`. The system books are `nc81base.sbk`, `nc81recs.sbk`, `nc81test.sbk`, `nc81trnr.sbk`, and `NC81BANK.SBK`, with `nc81int.dll` and `nc81rec.dll` beside them. Removing the system books leaves a blue screen, breaks resize, and stops the menu. The lesson file itself still opens. Renaming `nida81.tbk` removes the splash and reports that the shell is not a book. The lesson can still be entered.

`9e954d01.tbk` is the book every later measurement is checked against. It is Logic Families. At file offset `0xB1A96` it names `tb80r.sbk`, `nc81trnr.sbk`, `nc81base.sbk`, `nc81recs.sbk`, and `nc81test.sbk`, and it calls `initializebase`, `initializerecs`, `initializetest`, `openSession`, and `gotoPageList`. It also jumps to `nida81.tbk` and to `93954302.tbk`.

Who used it

Known. The published users of this particular set are the Navy lessons on disk and the training shops that shipped ToolBook courses. This is not a living authoring market. Someone outside the set who still holds a native 8.x book has the same file problem. That last sentence is a thought, not a census (WV-030).

Machines

The floor for a real run is Windows 98. That is the known-good debug host, not the top of the range. The ceiling is 32-bit Windows XP. The 16-bit NE images do not run on 64-bit Windows. A correspondent described 8.1 as 32-bit and 9.0 as the release that added 64-bit support. The binaries in this archive are 16-bit NE. Until a 32-bit 8.1 engine is actually in hand, the binaries win (C-006).

SoftICE work is closed as of 11 October 2026. Later tracing used the Watcom debugger on a fresh Windows 98 image. Those two debuggers do not share commands. See chapter 7.

2. File formats

Claims

Id	Grade	Statement	Source
F-001	Known	A lesson begins with a `0xC00` PE stub that has no useful code. The book stream is marked by `ZZZTBKBOOKLOC` and magic `0x22222222`, then `03 4A 42 4F` and version `01 00 00 08`.	WV-002
F-002	Known	Page records are the only objects that start `80 65 00 05 00 02 02`. Widgets inside a page start `14 00 00`.	WV-003
F-003	Known	`.tbk` and `.sbk` are the same container. The page name sits at offset `0x0B` of the page record.	WV-031
F-004	Known	Field text is a `Text\0\0` property: 8 bytes, a 16-bit length, then the string. The string often starts with CRLF.	page reader notes
F-005	Known	A ToolBook 11.5 Save To XML, enabled by `saveSpecial=true`, writes a folder of XML plus resources. A `.tbkx` file is a ZIP of that folder.	`TBdecompile/trail/20261014/03-tbx.md`
F-006	Known	Binary 8.1 stores many draw objects as bounds. The 11.5 XML expands those bounds into Vertices lists.	P1040 pin bounds `5055,4605,5205,4860` match the TBX Vertices start
F-007	Known	Foolbook `.fbk` / `.fsk` is a 4-byte magic (`FBK1` / `FSK1`), one flags byte, then JSON. Bit 0 means the payload is zlib-compressed.	Foolbook format notes

2.1 Native book

`tbload.exe`, through `MINSTB32.dll`, searches for `ZZZTBKBOOKLOC`. The signature check is a virtual call, not a literal compare sitting in `tbload`. On `9e954d01.tbk` the JBO body is at file offset `0xC00`. A later open measurement put the same book's first useful read at `0x9EC`. Both figures are about the same file. `0xC00` is the stub length. `0x9EC` is where one host's stage-2 open landed. Do not treat them as two different formats.

A page record holds a 32-byte name, usually `P` plus digits, then a property block. The property block at +43 of the early notes starts `14 00 00 01`. Widgets are not separate top-level objects on disk. They are length-prefixed chunks nested in the page.

Known. An ascending table of words inside a page, such as the 27 words at page `+0x8C3` on P1170, is an offset directory into a `14 00 00 83` content blob. It is not a Group. A Group is its own object and has its own member list.

Not true (WV-010). On-disk `14 00 00 01` does not become `14 00 00 11` when the book loads. One early dump was read that way. Later live objects used `14 00 00 02`. `9e954d01.tbk` has both `01` and `11` on disk. `93950101.tbk` is the other way around. That is a file difference, not a load-time mutation.

Property ids recovered from `TB80BAS` resource type `0x2D1`, id `0x2D0`, are `0x4000` plus a slot:

Id	Name
`0x4004`	bounds
`0x4015`	idNumber
`0x4023`	script
`0x402E`	text
`0x4034`	vertices
`0x4050`	Point-array setter. Runtime only. Not a record marker on disk.
`0x4059`	layer
`0x4097`	normalGraphic
`0x4098`	invertGraphic
`0x4099`	disabledGraphic
`0x409A`	stretchGraphic

Id	Name
0x409B	checkedGraphic

The graphic ids are the names on a Button in the TBX specimen (`bitmap id N`) and in the property resource. `N` is a resource index, not a file offset. Which bytes inside a `14 00 00` chunk hold `0x4097` is still open. An early carve that reported 641 BMP headers is not this binding. The count used against the specimen is about 75 BMPs in the export, and 67 markers paired with 72 of them.

2.2 System books

`.sbk` files use the same container. The NIDA `.sbk` files are resource containers: the object-type name table (ComboBox, Stage, Picture, Hotword, RecordField, PaintObject, Curve, AngledLine, Pie, Arc, IrregularPolygon, Polygon, Line, RoundedRectangle, Ellipse, Group, Field, Button, Rectangle, then Page, Background, Book) and menu resources. They are password-protected. The stored ciphertext and the empty-key decode are chapter 16.6. The decompile does not need the password to read the records.

Thought, narrowed. A search for the plain words `to handle` finds no OpenScript in those `.sbk` files. A later reader found compressed handler streams in `tb80r.sbk` (33 handlers), stored as keyword opcodes rather than the words `to handle`. “No script at all” is too strong. What those streams do when a lesson runs has not been traced. `nc81base`, `nc81recs`, `nc81test`, and `nc81trnr` still have no decoded `to handle` text.

`nida81.tbk` is a real book, not a resource bag. It has 43 objects and pages `P20000` through `P22130`. It does not name lesson ids. It has 100 `buttonClick` handlers in the same bytecode as a lesson. P310 is Minimum Requirements (586, 133 MHz, 12 MB RAM, 1 GB, mouse, Windows 98). P20001 is the small menu. P22130 is Test Console Setup.

2.3 The 11.5 export

Known, as of the exported specimen. In `Instructor115.ini`, under `[Options]`, set `saveSpecial=true`. Open a copy of the 8.1 book, answer the version warning, press F3 for Author level, then File → Save To XML. The result is a folder named `<book>.TBK TBX`. A `.tbkx` is a ZIP of that folder. Opening the ZIP in 11.5 imports it and rebuilds a book.

The schema says Bounds is required on a draw object and Vertices is optional. The schema spells IrregularPolygon wrong (`IrregularPloygon`). The binary stores bounds. The export expands them to Vertices. On P1040 the pin bounds `5055, 4605, 5205, 4860` match the start of the TBX Vertices list. That is the expansion direction: bounds in the 8.1 file, point list in the XML.

The exported specimen of `9e954d01` has 131 `Page.xml` files, 14 backgrounds, about 75 BMPs, and 302 `.script` files. Two of those pages, P440 and P2920, have no `80 65` page record of their own. The binary has 129 pages. All 129 names match a TBX page. They are not a missed page marker. P440's sentence and its house shapes are inside the P430 record (chapter 13). P2920 has not been searched the same way.

Saving from the 11.5 trial converts the book. An 8.1 runtime cannot open that converted file. 8.1 stores an imported picture as a basic blob and turns it into a BMP. 11.5 stores WMF, JPG, TIF, BMP, PNG, and GIF as themselves.

WV-009 called the export a conjecture on 12 October. The specimen exists. The conjecture is closed. WV-013’s description of the ZIP-plus-XML shape matches the specimen.

2.4 Foolbook envelope

`.fbk` and `.fsk` are not renamed JSON files with a new extension. The envelope is magic, one flags byte, then a JSON payload. The engine reads compressed, uncompressed, and the older plain-JSON working copies. `.json` remains the working copy. `.tbk` and `.sbk` stay supported so an existing ToolBook file can be opened. System books load core first, then user `.fsk` files as mods.

The philosophy written for that reader is discipline with a recorded deviance: use explicit points when they exist, then the TBX Vertices, then a bounds approximation, and say which one was used.

3. Runtime

Claims

Id	Grade	Statement	Source
R-001	Known	The intended launch is `TB80RUN` (32-bit) spawning `TB80NET.EXE` with the book on the command line. The seam is that spawn, not an in-process thunk.	module catalog, `TB80RUN` survey
R-002	Known	`TB80RTM.EXE` is the 16-bit shell. It owns `TBKMain`, `TBKClient`, `TBKChildWindow`, `TBKReserved_1`, `SchedulerWndTB80`, and the frame class created with `TB80MAIN`. A lesson click is delivered to `TBKClient`.	SoftICE class list
R-003	Known	`TB80MAIN.EXE` is a 23552-byte 32-bit stub. It calls `TB80RMT CreateRemoteCallServer` and pumps keys. It is not the book engine. Failure text: “TB80MAIN: Loading ActiveX support failed.”	`TB80MAIN` decompile
R-004	Known	`TB80BAS` is the book server (“ToolBook Server DLL”), 606 exports. `CDBINIT` is ordinal 1. `CDBFILEOPEN` is ordinal 8. `CDBVIEWPAGE` is 15. `CDBVIEWBKGND` is 18. `CDBPAINTCLIENT` is ordinal 26.	export table, WV-011
R-005	Known	`TB80RTM` `WM_PAINT` calls `CLIENTTOPAGE`, then ordinal 26. Shape drawing is in BAS, not in RTM.	`tbwp0018/rtm-paint-path.md`
R-006	Known	`TB80RMT` is not on the path that opens a lesson. The 1.5 `PHYSSIZE` index is not in these 8.1 books.	WV-014
R-007	Known	`nc81int.dll` exports 47 trainer calls. `IntCheckComm130E` opens a port at 9600 8O1, sends a probe, and returns `0x3D` on match or `0x3C` on failure. Families: 130E, TD2000, 5050, 205E.	`nc81int` exports
R-008	Known	`nc81rec.dll` writes `0x46`-byte session records to `SESSION3.REC`. Exports include `RecStartSession`, `RecCloseSession`, `RecBookmarkSession`, `RecWriteResponse`, `RecReadVariable`.	`nc81rec` exports
R-009	Known	On 2026-10-16 the `TB80RTM` exports were broken before the run, and again after one step once the program was running. Neither pass fired. The run went from start to end. At the end the Assembly window was `0000:0000` and `????`.	that session
R-010	Thought	Those symbols were not bound to the code that executed. "Set too early" does not cover the second pass. Why a break set after one live step still missed is not known.	R-009

3.1 Process

`tbload.exe` finds the book marker and hands off through a vtable at `FUN_00406a70` (virtual call at `00406aa1`, vtable+4). That vtable is not resolved. Thought: that call is the first handoff out of the 32-bit search.

`TB80RUN` does not open the book itself. It starts `TB80NET.EXE`. `TB80NET` is a 16-bit NE executable, not a DLL. It is the network access provider (zlib, `TBKNET3WNDPROC`, `TBK.INI`). It does not parse pages.

TB80W32.DLL (about 246 exports) and TB80W16.DLL (about 406) are the thunk halves. W16 is also the DoCmd* command bus (DoCmdDraw, DoCmdMouseDown). TBFILE32.DLL supplies long filenames to the 16-bit engine. TB80F32L.dll is the flat 32-bit entry and the common-dialog hook.

TB80RTM is the window. Classes seen live include SchedulerWndTB80 and TbkNet3TB80 (the latter owned by TB80NET). The lesson click does not arrive on TBKChildWindow. Break the TBKClient window procedure. BMSG does not see that 16-bit queue.

3.2 Book server and neighbors

Module	What it is for	What it is not
TB80BAS.DLL	Objects, properties, painters, twip scale, file open	Not the window
TB80CMP.DLL	OpenScript compile and expand. Resources 0x77 and 0x78	Not the page walker
TB80UTL.DLL	Type-name table, string manager, metrics, heaps	Not the 14 00 00 reader
TB80RCR.DLL	Linked beside the book server. Exports only partly named	Not classified
TB80ENU1.DLL / ENU2	English strings, 16-bit and 32-bit	Not code that opens a book
TB80BMP.DLL	DIB, metafile, FX blit. Ordinal 3 is the bitmap reader	Not a picture store
TB80JPG.DLL	JPEG. Version resource still says TB60JPG.DLL	
TB80LNL.DLL	MCI, wave, MMIO. The notify callback is Lionel	
TB80MM.DLL	tbkmmNotify, imageNotify, mciNotify, timerNotify	Does not classify objects
TB80OLE / OCX / ACX / VBX	Clipboard, controls, automation	Not on the Logic Families paint path

linkDLL libraries add commands only when a handler links them: TBDLG, TBWIN, TBDOS, TBREG, TBDB3, TBPDX, TBCBT. TBCBT is the CBT scoring extension. Its segment 7 holds the password cipher. See chapter 7. TBWIN is INIT_TBKWIN and TBM_EVALUATE. Neither TBWIN nor TB80NET walks page records.

Author binaries TB80EDT.DLL and TB80.EXE are not in this set. The runtime half of the author contract is visible: BAS exports CDBEDITBOOK, CDBVIEWBOOK, CDBISVIEWERINGENERICEDITMODE, CDBSENDEDITLEAVETOALL*, and TBXAUTHORMODE. RTM calls GETAUTHORLOCALIZEDDLL. That is bookmarked, not solved.

3.3 NIDA layer

A system book adds script above the book. A linkDLL adds commands. Most of what an author treats as “ToolBook” is one of those two, not a new entry point in BAS.

tb80r.sbk is the Asymetrix reader book (Help, Bookmarking, Logging, Path manager, Popup Page Manager). It is not the engine. tb80hyp.sbk is hyperlink and help. tb80anm.sbk is path animation.

The lesson calls initializebase, initializerecs, and initializetest by name. Thought. Those names have not been watched as a call log, and the order in toolbook.lst has not been watched live. version.ini in the set is 03.001.0006, dated 04-18-05. CAILT.DAT maps course ids to Navy CDs. 9E954 maps to DIE. STDAPPS.LST is a short encoded list, not a book.

nc81int.dll and nc81rec.dll are 32-bit PE. They are not Asymetrix. The trainer probe and the session file are the two behaviors that matter for getting a NIDA lesson to run as a lesson rather than as a picture of a page.

3.4 What startup has been watched

WDW on W16, before any book open, hits platform detection, OLE init, and the 32-bit thunk: GETPLATFORMREF, IS32BITENV, GETRUNTIMELOCALIZEDDLL, ACXINITIALIZETOOLBOOKSUPPORT, ACXOLEINITIALIZE, LOADLIBRARYEX32W, GETPROCADDRESS32W.

WDW on UTL, after INITTASK, hits string and task utilities: GETSTRMGRSTRING, plugin init, TBK.INI profile reads, metrics, GHEAPCREATE. No record walk.

The Unicorn host has not opened a book. Pass 06 asks for TB80.INI and exits. See chapter 7.

3.5 The RTM export list, and a dead task

Watcom's Functions window for TB80RTM is 43 exports. ?CALLPROC32W@@Z at the top of that window is the decorated CallProc32W import. The Ks are Watcom's size marks, not part of a name.

EGYPT is the old window prefix. The frame is MAINWNDPROC. The client is EGYPTCLIENTWNDPROC, the child is EGYPTCHILDWNDPROC, and EGYPTNOTIFYPROC takes the notifies. STATUSBARWNDPROC is the status bar. SUBEDITCONTROLPROC subclasses an edit control. DDEWNDPROC is DDE. XDLLTRANSWINPROC is a window owned by another DLL.

The shell's own book calls are ENUMWINSFOROPEN, BOOKQUERYSAVE, EGCDBSAVEBOOK, and HANDLEREMOVESCRIPTPROC. EGDYNALINKERRORFUNC runs when a DLL did not load.

Around that frame the names are the product: ASCII file dialogs, cache-file and CD-ROM save, optimized-save progress, the image store, a palette broadcast, the print dialogs, find and its condition dialog, two ask dialogs whose PL stem is still unread, resource pickers, TOOLNOTIFY, and the remote pair XCTOLEREMOTECOMMAND / XCTOLEREMOTEGET.

The 12 October export map gives each name a line in Toolbook/_decompiles/TB80RTM.EXE.c. A first pass took every call in the next 400 lines and credited it to the export. That mixed in the next function. The following is only what is inside the export itself.

EGYPTCLIENTWNDPROC switches on the message. 0x0F is paint: FUN_1070_0be0, or GETUPDATERECT, or FUN_1060_0dca, depending on the flags at the window struct +0x124 and +0x126. 0x14 returns 1, so the default background erase does not run. 0x02 calls ordinal 76. 0x07 and 0x08 are focus. 0x100, 0x101, and 0x102 are key down, key up, and char. 0x104 and 0x105 return without acting when DAT_1450_0a41 is set. 0x113 is the timer. 0x4A is WM_COPYDATA. Help ids 0x7DF9, 0x7E02, and 0x7E03 go through CXTHELPSETCONTEXT.

MAINWNDPROC is 57 lines. The message is private: 0x41, 0x4AA, 0x4AB, 0x4D8, 0x4D9. 0x111 is WM_COMMAND. A fixed set of command ids goes to FUN_1098_0124, and the rest to FUN_1368_0000.

BOOKQUERYSAVE resolves the path with ASYM_GETLONGFILENAME when the process is 32-bit. If DAT_1450_0a41 or DAT_1450_0a45 is set it returns 7 and does not prompt. Otherwise it prompts through FUN_1068_0034 with style 0x31.

EGCDBSAVEBOOK is the same shape and it does not call CDBGETVALUEEX. An empty name, or either of those two flags, sets the result to 7. Otherwise it prompts with style 0x24 or 0x30 and returns whether the answer was 6.

HANDLEREMOVESCRIPTPROC is a dialog procedure. 0x110 sets the item text from ordinal 106. 0x111 with a zero id calls ENDDIALOG. EGDYNALINKERRORFUNC returns immediately unless the id is 0x1FAC or the _C_FILE_INFO_ id, then calls ordinal 161. POSTPALCHNGTOOTHERS is seven lines: SENDMESSAGE of 0x311, which is WM_PALETTECHANGED, and return 1. CHANGEPRINTERDIALOGPROC reads and writes the profile string. ASCIIFORMATDLGPROC is a dialog on 0x110 and 0x111, command ids 1, 2, 0x106B, and 0x106C. PLASKDIALOGPROC and PLREQUESTDIALOGPROC end the dialog. The PL stem is still not a word in the listing. The palette helpers TBKPALAPPEND and CREATEPALETTE sit in the next function, not in POSTPALCHNGTOOTHERS.

4. Objects, file open, and the reader

Claims

Id	Grade	Statement	Source
O-001	Known	A live object is `0x76` bytes. Bounds are four words at `+0x0F`, in twips. At 96 dpi, screen pixels are stored / 15, which is `(stored * 96 + 720) / 1440`.	live traces, `FUN_1008_01dd`
O-002	Known	`+0x1F` selects the BitBlt raster op. `+0x3B` is a GDI handle. `+0x30` is the point count. `+0x32` is the point-array far pointer, 4 bytes per point, stored twips.	polygon and BitBlt traces
O-003	Known	`CDBSENDCREATE` (BAS ordinal 51, `12c8:00BC`) creates or rewrites objects from the Instructor UI. The runtime book-open path does not call it. `FUN_12c8_0ef8` is downstream of that author path.	`tbwp0025/cdbsendcreate-is-author.md`, WV-042
O-004	Superseded	The chain `CDBSENDCREATE` → `FUN_12c8_0850` → `FUN_12c8_0e32` → `FUN_12c8_0ef8` is the runtime reader.	`TBdecompile/trail/20261015/00-how-toolbook-operates.md` §2, `tbwp0002`, `tbwp0008`
O-005	Known	SoftICE and WDW both reached `OBJFILEOPEN` and the buffered read in `FUN_1160_03de`. The raw DOS path at offset `04F3` is skipped when the buffer is already satisfied.	2026-10-16 consolidated note
O-006	Known	The W16, UTL, and BAS traces in `tbwp0026` do not contain a function that reads a `14 00 00` record. `CDBCHUNK*` walks text. `VALUENEWLONG` constructs a value. `OBJFILEOPEN` is downstream of the reader.	`tbwp0026/trace-review.md`, WV-044
O-007	Superseded	The runtime reader has no name and has not been watched.	open-question notes. The 20 October read is O-008.
O-008	Known	On a lesson open the object body is read by `1178:08B2` → `1188:04DA` → `1188:08D6` → `1160:02CC`. `HUGEGETDATA` is a different tree and was not on this path. Message `0xE` runs after the bytes are in memory.	Bochs, 2026-10-20. Section 4.6. Appendix L.
O-009	Known	A field's text handle is word 1 of the block behind object `+0x24`. On disk that word is a trailer-handle index, and the field class in the specimen is `0xA`. Word 0 is the style-run handle. `FUN_1440_0160` does not read `+0x42`.	XML match, and `research/store-20261022.md`.
O-010	Known	Class `0x3A` is the book's `ClassTbl`: `0x2C`-byte rows. Class `0x26` is `Viewer`. In the sample books the next free number is `0x3D`.	`research/class-table.md`.

Id	Grade	Statement	Source
O-011	Known	A class 5 page holds its background as a far reference at `+0x47` / `+0x49`. On `9e954d01.tbk` all 131 resolve to a class 4 record.	`research/store-20261022.md`.
O-012	Known	The ordered page list hangs off book `+0x10`. Each row is 13 bytes: background id, page id, page reference, and a zero.	Same note. 131/131 on the specimen.

4.1 The live object

These offsets are the ones the papers and the live traces agree on.

Offset	What	Grade
`-2`	Type byte. `0x11` is an arc. The other type bytes are not mapped.	Known for `0x11` only
`+0x0F`	Bounds: left, top, right, bottom. Written as twips.	Known
`+0x17` `+0x19` `+0x1B` `+0x1D`	Four words copied by `FUN_12c8_0ef8` from a source structure at `+0x3E` `+0x40` `+0x42` `+0x44`. The disk `14 00 00` record does not sit at those source offsets.	Known as a copy. The source is not the disk record.
`+0x1F`	BitBlt raster-op flag. An earlier note also said the bits are taken from a source structure at `+0x0A`, `+0x0C`, and `+0x0E`. That copy was not re-checked against the BitBlt trace.	Known as the raster-op byte. The source-bit copy is Thought.
`+0x30`	Point count (word)	Known
`+0x32`	Point array far pointer	Known
`+0x3B`	GDI handle	Known
`+0x52`	Per-pin index. Not applied by the current renderer. Pins that share point bytes draw on top of each other.	Known that the field exists. Not known how paint uses it.

`ScaleTwipsToScreen` is `FUN_1008_01dd`, BAS segment 2 offset `0x1DD`. With `DS:01A6` and `DS:01A8` set to 96 it turns stored twips into screen points. On the P1140 to P1130 change those words were 96 and 96, so the short form is division by 15.

`FUN_12c8_0ef8` copies four words into the object. It does not parse a page. The disk record and the structure it copies from are different shapes. The step between those two shapes is still unnamed. That step is not `CDBSENDCREATE`.

4.2 Author create, stated once

`CDBSENDCREATE` receives bounds that are already formed. It does not compare a buffer to `14 00 00`. Nothing inside BAS calls it by name. One external thunk from `FUN_13d0_0822` reaches ordinal 51. That caller is author UI state (it tests bytes such as `0x26`, `0x36`, `0x38` and stores coordinates around object `+0x638`). It is not the book-open loop.

Do not break on ordinal 51 to find the reader. The stop will fire when Instructor creates or rewrites an object. It will not fire because a lesson page was opened. The 15 October operational notes ended this section with “list the XREFs; the caller that passes a page blob is the reader.” That sentence is not true. The one caller that was found does not pass a page blob.

The function is still real. `CDBNEWOBJECT` indexes objects as `param_6 * 0x76`. The size matches the live object. The index expression is the decompile, not a second measurement of the size. The block that holds the array is `FUN_1278_3be6` (decompile line 121070): `GLOBALALLOC` or `GLOBALREALLOC` of `count * 0x76`. That is not `TBHEAPALLOC`. Chapter 19.

`param_5` selects the path. The string arithmetic in the decompile is Ghidra matching those constants against `s_CTbkSimpleOLEDispatch_CallMethod`. It is not a string compare in the program.

param_5	Path
0x1014, 0x1015	FUN_12c8_0850, then FUN_12c8_0e32 (allocator), then FUN_12c8_0ef8 (field copier)
0x1021, 0x1022	FUN_12c8_0a86
0x3018–0x301B	CDBCHECKBOUNDDTL, then FUN_12c8_0e08
0x3022	FUN_12c8_0d40
anything else	Ordinal 115, TOOLBOOKFATAL

param_3 and param_4 are the bounds pair. They are clamped against 32000 and 0x8300. param_6 and param_7 are compared to the current object id. param_9 is the error out-byte. FUN_12c8_0850 has one caller, uses ENTER 4,0, locks a context, and tears the object down on error. It is not the record reader.

FUN_12c8_0ef8 writes the four bounds words two ways. If its source pointer is null, it copies globals that were set from the parameters (DAT_1630_01FE through DAT_1630_0204) into +0x0F, +0x11, +0x13, +0x15. Otherwise the default arm of its param_6 switch copies four words from that pointer into the same slots. The same function copies source +0x3E / +0x40 / +0x42 / +0x44 to object +0x17 / +0x19 / +0x1B / +0x1D, a byte from source +0x06 through a type table, and source +0x04 as the pattern lock. The disk 14 00 00 record is not that source structure.

param_6 in that switch is an object type, not the param_5 above:

param_6	What the decompile does
0x0F, 0x10, 0x13, 0x14	Polygon-like. OBJNEWNEAR(count << 2) for the point array.
0x11, 0x12	Left alone. 0x11 is the arc that was seen live.
0x24	Copies four words from the source pointer, and does something else the listing did not name.
default	Copies the four bounds words.

The property registrar is FUN_1058_0a98 at 1058:0A98. It writes property ids. It does not write painter pointers. The ids it writes are 0x4004 (bounds, nine times; the first is decompile line 30313), 0x4009, 0x400D, 0x4015 (idNumber), 0x401A–0x401C, 0x4020–0x4022, 0x4023 (script), 0x4027, 0x402B, 0x4031–0x4033, 0x4034 (vertices), and 0x4035. Names are only attached where the property resource or the TBX specimen has attached one. The others are slots, not captions.

Older notes that drew this chain as the runtime path:

- tbworkingpapers/tbwp0002/bas-create-path.md
- tbworkingpapers/tbwp0008/cdbsendcreate-sites.md
- TBdecompile/trail/20261015/00-how-toolbook-operates.md, section 2
- TBdecompile/trail/20261012/01-object-create-path.md, the “still open” paragraph
- TBWorldview/ToolBook_Architecture.md, the book-open flowchart that calls CDBSENDCREATE per widget

Those files still describe a real function. They mis-place it. The tables above are the part worth keeping.

4.3 File open, as far as it has been stepped

CDBFILEOPEN is BAS ordinal 8. Thought (WV-007). It is the export on the open path. It has not been shown to expand a 14 00 00 record itself.

Under it, OBJFILEOPEN has a large frame (0x25A in one trace). FUN_1160_03de is the read wrapper. It calls FUN_1258_00a8 (buffered path). If that call satisfies the request, a jump skips the raw DOS read at offset 04F3. Live dumps of a 4-byte read returned an internal pointer (FC B5 BD B0 at one buffer), not a page header and not the JBO signature. A wider dump showed 4F 46 nearby. That is not a page record.

CDBCHUNKFIND, CDBCHUNKLAST, CDBCHUNKEND, and CDBSKIPSEP walk characters: comma, CR, LF, tab, space. They are text chunkers. A decompile that shows ISDBCSLEADBYTE and a comma test is not the page walker.

4.4 Where the reader is not

Closed, so they are not searched again as the reader:

Candidate	What it actually is
`FUN_1028_2680`	Filter registrar. Calls `TBREGISTERFILTER`. The `0x14` is a type-code limit, not the record marker.
`FUN_1008_2d31` (switch at `1008:2DC7`)	OpenScript math. `+ - * /`, `abs`, `sqrt`, `log`, `exp`, `sin`, `cos`, `atan`, `asin`, `acos`.
`TB80RTM`	No named file open. Paint goes to BAS ordinal 26.
BAS segments `1018`, `1278`, `1478`, `14b8`, `1508`, `10d0`, `1030`, `14a8`, `1070`	Sampled. No stream walker.
`1160:03DE`	The buffered read seen on 16 October. The lesson-open body read is `1160:02CC`, section 4.6.
`FUN_10b0_00a6`	Dispatches on type codes including `0x14`. It is a type dispatcher, not a reader.
Code at `12e8:0000`	OLE object clone.
BAS segment 182	Real preload code, 39 far calls with a zero segment and no reloc. Not on the book-open path.

Known. No static search of the disassembled image has found the bytes `14 00 00` used as a compare against a far pointer that then builds an object. Thirteen undisassembled runs contain the word `14 00`. Those are far-call immediates, `MOV CX, 0x14`, or jump displacements.

Known. The 20 October read does not compare the marker. It uses the length in the handle-table entry. The marker is still in the bytes, at record offset 50 in the traced object and at 52 in 146 of 206. Section 4.6.

4.5 Page and background

Known. `80 65` records are the pages. `9e954d01.tbk` has 129 of them.

Known. A TBX page can name a background by an id, such as 23 on P10213, that does not appear as a word or a dword in the matching binary page. The on-disk link is still not matched. Parent and z-order are not in the page record that has been decoded. Absolute bounds are not stored for every shape. The P1170 bracket was computed from bounds relative to a parent. The parent offset has not been applied to the P420 rects.

Known, live record only. A class 5 page holds a far reference at `+0x47` / `+0x49` (O-011). Property `0x4016` asks that object for its size. Property `0x429d` is forwarded when the reference is nonzero. The class 4 record has the fill, the pattern, and the size. Property `0x4036` returns only the low word. Follow the dword. The note is `research/paint-20261021.md`.

4.6 The 20 October read

Bochs, Win98 SE, debugger outside the guest. Appendix L is the command card. Selectors in this section are that boot only.

Known. `1048:2174` opens the book. Each object then hits `1178:08B2`. That calls `1188:04DA` (allocate `header + body`) and `1188:08D6`. `08D6` calls `1160:02CC`. The traced call read `0x19C` bytes from file position `0xB7221` into the block at offset 6. A flag `0x40` on the entry caused one more read, 2 bytes, at `0xB73BD`. `HUGEGETDATA` was not hit.

Known. The on-disk table is 206 entries of `0x18` bytes at file `0x10D6`. Index `0x12` is this record: position `0xB7221`, length `0x19C`, type byte `0x25`. Handles `0x0C1E`, `0x0C36`, and `0x0C4E` are `0x18` apart in memory. A 20-byte dump of the live entry matched the file through `+0x13`, except status (0 in memory, 1 on disk). Bytes `+0x14` through `+0x17` were not dumped. On disk those four bytes are `00 21 25 00`: a zero, the nibble-swapped index, the type, and a zero.

Known. After the read, `1178:08B2` sends message `0xE` through `1180:02CE` and `1180:03C8`. Handle `0x0C1E` returned `10B0:0000`. There, `0xE` calls `FUN_10b0_54e0`, which calls `FUN_10c8_018e` only when the error byte is still 0. That is a near-handle fixup, not the file read. Handle `0x0C4E` returned `10F8:0000`, which has no `0xE` case. The handler for

handle 0x0C36 (this 412-byte record) was not captured. On the object that was dumped, message 0xE changed only block bytes 0 through 3, to a far pointer to the reader object. The 412 body bytes did not change.

Thought. Record byte 12 matches the table's type byte in 142 of 206 entries, so it is the class code. The 0x12 seen on the 03C8 stack was a different object. It is not yet that byte.

4.7 Scripts and groups

The detail is research/paint-20261021.md. These are live-record offsets.

Known. FUN_10d0_12e0 picks the script by the class word at record −2. Book (class 1): mode in the low two bits of +0x22, word at +0x24. Page and background: mode +0x30, word +0x2A. An ordinary object: mode +0x0A, word +0x0B. The top bits of book +0x22 are the color-screen flags. Mode 0 is inline. Mode 2 is a type 9 resource. The shared script is a separate request, type 0x13. Modes 1 and 3 have no branch.

Known. A group's count is +0x24. The entries at +0x2E are 10 bytes: the child, then its left, top, right, and bottom. No separate stream-in was found. Whether load rewrites the child word was not checked.

5. Graphics

Claims

Id	Grade	Statement	Source
G-001	Superseded	Polygon kind 04 00 with an inline point list at +0x22.	The specimen's class 0x10 keeps the count at +0x30 and a near pointer at +0x32. G-002. research/store-20261022.md.
G-012	Known	A resource id is the fourth word of the 8-byte directory entry. A button stores that id plus one, at graphics-block +0x15.	FUN_1128_1e44 accepts a match only when the scanned word is the fourth. 396/396 buttons.
G-002	Known	The polygon painter is BAS FUN_1408_0376 (segment 0x82 offset 0x376). It copies object +0x30 / +0x32 to DS:30AC and calls GDI!POLYGON.	SoftICE, 2026-10-10
G-003	Known	Content rectangles are bounds-only. In the page blob the tail is four twip words, a zero dword, a fill dword, then 01 00 00 04. 9e954d01.tbk has 961 of that marker.	Foolbook shapes.py count, P1040
G-004	Known	An arc object is type 0x11. Four twip pairs at +0x25, +0x29, +0x2D, +0x31. Bounds at +0x0F. Fill at +0x18. Painter is BAS segment 0x7B.	live arc, 2026-10-10
G-005	Known	ToolBook does not call GDI!PIE. Pie wedges are computed by GDIELLIPSEPOINTFROMANGLE (property 0x4123) and filled with the arc geometry.	import table and decompile
G-006	Known	A diagram bracket on P1170 is not stored as points. BAS segment 0x5B computes four polyline points from the object bounds.	live GDI!POLYLINE, 4 points
G-007	Thought	P420’s flame is several pen strokes in the record at page +0xA13, matching three 96-point red IrregularPolygons in the TBX export. The on-disk control structure is not known.	render attempts, TBX
G-008	Known	A live button holds its four picture ids in the block at +0x28. Normal id at block +0x15, invert +0x27, disabled +0x39, checked +0x4B.	research/paint-20261021.md
G-009	Known	A stored bitmap is an 8-byte prefix, then a Windows BMP. 9e954d01.tbk has 67. Use bfSize from the BM header. The prefix dword is not always bfSize + 8.	same note, file scan 2026-10-21
G-010	Known	Resource directories are one per type. FUN_1028_218a reads OBJGETHDRLONG((type + 5) * 4). Type 1 is a bitmap. TB80BMP is not the lookup.	same note. One Bochs hit returned an object reference for type 9

Id	Grade	Statement	Source
G-011	Known	Draw-object pattern `+0x25`, stroke `+0x17`, fill `+0x1B`. Background pattern `+0x4B`, fill `+0x5B`, size `+0x5F` / `+0x61`. `0xFD` is hollow, `0xFE` solid paper, `0xFF` solid ink.	same note

5.1 Who paints

The painters live in `TB80BAS`. They are reached through a type dispatcher, not by a named call from outside BAS. `TB80RTM` handles `WM_PAINT` only as far as `CLIENTTOPAGE` and `CDBPAINTCLIENT` (ordinal 26).

Painter	Where	GDI
`PaintPolygon`	`1408:0376`	`POLYGON`
`PaintPolyline`	`13e8:00E6`	`POLYLINE`
`PaintRectangle`	`1020:03E4`, and segment `0x83` (`FUN_1410_0066`)	`RECTANGLE`
`PaintArc` / alt	`10A0:0558`, `10A0:0458`	`ARC`
`PaintEllipse`	`1230:2F1C` and an alt	ellipse
`PaintRoundRect`	`1230:2278` and alts	round rect
`ScaleTwipsToScreen`	`1008:01DD`	none. Converts in place.

`FUN_1008_01dd` is that converter. For each pair it computes `(coord * factor + 0x2D0) / 0x5A0`. `0x5A0` is 1440, one inch in twips. `0x2D0` is 720, so the add is rounding. The x factor is `DAT_1630_01a6` and the y factor is `DAT_1630_01a8`. At 96 DPI the factor is 96 and the division is the same as dividing by 15. The `/15` rule is that case, not the function.

The polygon type table is not in the static image. Unpacked BAS still carries Optloader segment-table debris. The HTML listing has zero cross-references to `FUN_1408_0376`. Thought. The table is built at load, or fixed up by an NE relocation that the listing does not show as a pointer. A break on the painter, then one frame up, is how the table gets found. Searching the file for the bytes `76 03 08 14` will not find it. The selector is assigned at load.

`SetPointArray` (`FUN_1408_00d8`, property `0x4050`) stores the count at `+0x30`, allocates `count * 4` with `OBJNEWNEAR` into `+0x32`, and copies the points. That is the runtime setter. The file does not contain a `0x4050` record.

`LOCKPATTERN` (`1098:0876`) takes a pattern index at `[BP+0xC]`, accepts 1 through `0x80`, and calls `TOOLBOOKFATAL` outside that range. `UNLOCKPATTERN` is `1098:09FC`. It is a fill hatch, not a picture id. An earlier listing also saw a pattern index at object `+0x17` and `+0x5B`. `+0x17` is already the four-word copy in chapter 4. Do not treat those two notes as the same field until both are dumped on one object.

The October 12 field card puts the polygon's object pointer at `[BP+4]`. A far call's first argument, after `ENTER`, is `[BP+6]` (the manual's assembly card). The card did not record whether `PaintPolygon` was entered by a near call or a far call. Dump both slots once. The count at `+0x30` and the points at `+0x32` do not depend on that.

5.2 What is stored

Polygons. `9e954d01.tbk` has 96 kind-`0400` records on 13 pages. P430 has 22. A broader search that ignores the kind word produces false positives. P430 draws as a chip: silver pins (`0xC0C0C0`) and a light-gray body (`0xE0E0E0`). One live 11-point polygon is a pin tip, not the body. The captured screen points were `(218,335)` through `(218,341)`, a 10 by 17 notch. The same points exist in the file as twips near page offset `0x9CB` of P430: `(3147,3862)` through `(3147,3973)`.

Pins that share identical point bytes stack on top of each other until the index at `+0x52` is applied. That index is not applied in the current clean-room draw.

Rectangles. P1040 is the TTL intro page (TBX page id 880). Two DIP packages: 7400 on the right, 5400 on the left. The 8.1 file stores 28 pin boxes (fill `0xC0C0C0`) and two dark bodies (fill `0x404040` in the TBX, `64,64,64`). The pin

tip is not an 11-point list on this page. Paint expands the tip. The TBX export writes the expanded Vertices. Matching a TBX Vertices list back to the binary means matching the bounding box, not searching for 11 points.

Brackets and strokes. On P1170 the bracket's live polyline was `(354,201) (321,201) (321,238) (358,238)`. The object bounds in twips were left `4801`, top `3015`, right `5370`, bottom `3570`. Divided by 15 that is the same rectangle. The points are computed. Connecting lines and detail strokes of this kind are not extracted yet.

Arcs. Live geometry is known (G-004). Fill `0x808080` was seen at `+0x18`. No page record has been matched to type `0x11`, and the start and sweep angles are not located in the struct. Do not set a breakpoint on `GDI!PIE`. The call is not imported.

Curves. Curve's type id in `nc81base.sbk` is `0x05C7` (file offset `0x1810`). That id is not in the P420 page records. The point runs sit in the record at page `+0xA13`. A naive walk draws a tangle in screen space around 400–500. The TBX export of the flame is three 96-point red IrregularPolygons. Thought. The tangle is strokes concatenated that should be drawn separately. A length prefix or a zero terminator between groups has not been proven. There is no `Curve` name on the page.

Pictures. A button's four picture ids are in the block at live record `+0x28` (G-008). The bytes are an 8-byte prefix and then a BMP (G-009). `9e954d01.tbk` has 67 of them. The lookup is the per-type directory in G-010, not `TB80BMP`. A picture that is not a button still has no located id. `DRAWPICTURECALLBACK` writing a GDI handle at object `+0x0C` / `+0x0E` remains a reading of the listing, not a live dump. An earlier note said 641 BMPs carved. That count is a header scan, not a widget binding, and it is not the number to use.

The shared near-start graphic in the lesson books is a 581 by 157, 8 bpp gold plate, not a text banner. Two notes locate the `nida81` splash building: file `0x413FAB` (323 by 344, 8 bpp, off-bits 310) and a carved copy taken from file `0x487BE` (27874 bytes). This record does not treat those two offsets as the same byte. The live `BITMAP` header that was found was width `0x143`, height `0x158`, 8 bpp, stride `0x258`, `bmBits` null. Known that the splash is a DIB. Not known which property on the page object holds its id.

BAS segment `1150` builds the book palette (`CREATEPALETTE`, `SELECTPALETTE`, `REALIZEPALETTE`). An 8 bpp BMP needs that palette. Segment `1018` creates and deletes pens, brushes, and bitmaps. Who owns the lifetime is open. Fill color and stroke color are known (G-011). Pen width is not.

Field text is `DRAWTEXT` from a paint context at `param_5+0x4A`, `+0x4C`, and `+0x50`. The disk handle is word 1 of the block at object `+0x24` (O-009). Word 0 of that block is the style-run handle. The rest of the field layout, including the border, is open.

5.4 Classes, from the 21 October listing

`FUN_1058_2ae4` registers the draw segment. This does not retire G-002. The note is `research/paint-20261021.md`.

Class	Draw segment	Record	What it calls
8	`0x1410`	`0x28`	`FUN_1410_0066`, `RECTANGLE`
`0xC`	`0x13d8`	`0x28`	`FUN_13d8_0066`, `ELLIPSE`
`0xD`	`0x13e0`	`0x2C`	`FUN_13e0_015a`, `ROUNDRECT`. Corner radii at `+0x28` and `+0x2A`

The ellipse previously listed in segment `1230` is not this registration. Bounds for all three are `+0x0F`, `+0x11`, `+0x13`, `+0x15`. Class `0xF` is registered with a `0x34`-byte record. It is not yet named polygon.

5.3 What a render is allowed to claim

A clean-room picture of a page is not the engine (WV-005). The current honest draw is:

1 Explicit point records (kind `0400`) where the count and the pairs are in the file.
2 TBX Vertices where an export exists and the binary only stored bounds.
3 A bounds rectangle where neither points nor an export exist.

Say which of the three was used. P430 from kind `0400` is a check against the chip. P1040 from bounds is a check against the two packages, with the understanding that the pin tips are fatter on screen than the stored boxes. P420 from a raw point dump is not a flame until the stroke breaks are known.

6. OpenScript

Claims

Id	Grade	Statement	Source
S-001	Known	Handlers are compressed bytecode. The keyword table is NE resource `0x77` (type 100, id 119). The offset table is at byte 2 plus count times 4. There are 994 keywords.	decoder notes
S-002	Known	Bytes `0xC0`–`0xFF` copy from the compressed input stream, not from the output. An earlier decoder that copied from the output was wrong.	WV-032, decoder fix 2026-10-10
S-003	Known	The decoder used for NIDA navigation stops at the first standalone `end`, so trailing decode garbage is dropped.	engine notes
S-004	Known	`9e954d01.tbk` yields 76 handlers on 39 pages. `nida81.tbk` yields 100 `buttonClick` handlers. `tb80r.sbk` yields 33, with the `to handle` prefix also compressed.	corpus counts
S-005	Known	`FUN_1008_2d31` is the expression evaluator. The modern engine implements `+ - * /` and `abs`, `sqrt`, `log`, `exp`, `sin`, `cos`, `atan`, `asin`, `acos`, and otherwise leaves the original text.	`openscript_analysis` 1.0.0r6
S-006	Known	A `buttonClick` on `nida81.tbk` page `P20000` goes to `P20001` because the handler says so. Navigation is not “only a quoted page id.”	WV-033, WV-041
S-007	Thought	A short script compiled by Instructor 8 and expanded with the existing decoder would lock the keyword map. That compile has not been done.	`tbwp0014/instructor8-compile-test.md`

6.1 What the bytecode is

Resource `0x77` is a compiler emit routine as well as the keyword pool. Resource `0x78` (id 120) is another emit routine. `COMPILERPROPINFOFROMINDEX` locks resource `0x70`. The property-name table is a different resource (BAS type `0x2D1`, id `0x2D0`). Do not treat every locked resource in that range as the keyword list.

The byte classes, the back-reference formula, the corpus counts, and the handlers that have been expanded are chapter 14. The working description, including the two bugs and the compile test that has not been run, is Appendix J of the reference manual. An early reading of the same bytes produced "italic" and "systime" and concluded the keyword table was wrong. That reading copied `0xC0`–`0xFF` from the output stream and used the wrong offset-table stride. It is superseded. Do not cite it.

High-bit opcodes (`0xE0`, `0xF0`, `0xFF`, and neighbors) emit whitespace, CR/LF, or a repeated span. After the input-stream back-reference fix, the 76 handlers in Logic Families decode. Examples that are real source, not Latin-1 of the bytecode:

- `send next`
- `go to page "P400"`
- `go to page "P310" of book "nida81.tbk"`

The common subset the courses actually use is `go to`, `send`, assignment, `put into`, `set to`, `get`, increment and decrement, nested `if` / `else` / `end if`, `step from` / `to`, and `name of this book` / `this page`. The engine stops at the first standalone `end`.

`tb80r.sbk` uses the same bytecode. The difference is that the handler name prefix is compressed too. Encoding rules match. The byte histogram has more `0xFF` and `0xE0`.

6.2 What a lesson does with it

Logic Families is a menu of jumps, not a self-running program.

Page	What the handler does
P900	Lesson menu. TTL → P1001. Other → P2001. Final quiz → P10001. Exit → `nida81` P20001. No Next.
P1001, P2001	Back returns to P900, not to `nida81` P20001.
P300	`selection1` is the IBM PC Compatible Computer line. `buttonClick` sends `lastbook nida81.tbk` and `nextpage` P310.
P10001	Assessment. Directions → `nida81` P10020. Exit → `nida81` P20001. Begin → P10210.
P10410	Section end. Menu returns to P900.
P1290	Quiz. Correct choice 5408. Again → P1292. Next → P1300.

A retry page returns to the quiz whose `again` field names it. `initializeRecs` on the Maybe2 shell clears responses when a session loads. That behavior is inferred from the shell script, not from a live `nc81rec` trace.

The modern engine also accepts `send to`, a `get` that calls, `forward`, `break`, and `property of`. Predicates such as `is not in` are recorded rather than evaluated. Property writes are not applied to a live object. The TBX `.script` files are plain text and are easier to read than the bytecode. They are a playback path for the export, not a reader of the `.tbk`.

6.3 Expressions

The 148-case switch at `1008:2DC7` is `FUN_1008_2d31`. It stays in segment `1008`. It checks for names such as `log`, then uses the FPU (`FLD`, `FSTP`, `FCLEX`). It is not the record reader and not a paint path. The clean-room evaluator copies that set of operations and falls back to the original string when it does not recognize the right-hand side.

6.4 What is not done

The full keyword map is not locked. Resource `0x77` has 994 entries. The engine proves the ones the NIDA handlers use. A compile of a short script in Instructor 8, saved into a book, then expanded, would show whether the offset table and the names still match. That test is written down and not run.

Nested property expansion, the opcodes past the common subset, and `forward` into a system-book handler are unimplemented or only sketched. `COMPILERCOMPILETEXT` is CMP ordinal 1. `COMPILEREXPANDTEXT` is ordinal 3. `CFUNEXECUTE` is 8. `CFUNEVALUATE` is 9. Those ordinals are the stable breakpoints if the compile test is run.

7. Debugging and hosts

Claims

Id	Grade	Statement	Source
D-001	Known	SoftICE and the Watcom debugger do not share command syntax. SoftICE field work closed 11 October 2026. Later breaks are WDW.	field manual, `tbwp0026/wdw-break-syntax.md`
D-002	Known	WDW has no full automatic call trace. `break <symbol> resume on` plus `log <file>` logs and continues. `resume on` is a property of a breakpoint that already exists.	`tbwp0026/wdw-trace.md`
D-003	Known	In WDW, module qualification uses `@` (`TB80BAS@OBJFILEOPEN`). SoftICE's `!` does not work there.	same note
D-004	Known	Symbol maps are `NE-segment:offset name`. The segment index is not a live selector. SoftICE and WDW do not load the `.map` as a module.	`tbwp0003/TB80BAS.map` (3799 entries), `TB80RTM.map` (2669)
D-005	Known	The Unicorn host has not opened a lesson. Pass 06 exits after `TB80.INI`. The entry does not ask for the book.	WV-006, WV-036
D-006	Known	TBCBT NE segment 7 holds `KEYEDPASSWORDENCRYPTEX`, `MAKEKEYEDPASSWORDEX`, `ENCRYPTEX`, and `DECRYPTEX`.	WV-043
D-007	Known	NE segment numbers are not Ghidra's `CodeN` names. Segment 1 is `1000`, segment 2 is `1008`, and the usual step is 8, with gaps.	`TBdecompile/maps/rtm-ne-to-ghidra.md`
D-008	Known	From 2026-10-20 the live host is Bochs 3.1. The debugger sits outside the Win98 SE guest. SoftICE does not have to be loaded for a break.	Appendix L
D-009	Known	Offsets in the Ghidra listing equal offsets in the guest. Selectors do not. After each boot they are taken from the LDT and checked by segment size.	Appendix L, `maps/bochs-20261020.md`

7.1 SoftICE

The kit is `SI405w9x.exe` plus ICEDUMP 6.021. The loader that worked was `w9x/4.05.334/icedump.exe`. It loaded on the Windows 98 VM after both `kernel32.dll` and `user32.dll` exports were listed in `winice.dat`.

Ordinal breakpoints are stable across boots. Selectors are not. `TB80BAS.Ordinal_8` is `CDBFILEOPEN` on every boot. `36F7:03DE` is `FUN_1160_03de` only until the next reboot.

`/DUMP` writes the SoftICE console, not the address given, unless the dump experts are set. Several early `page.bin` and `rtm.bin` files are command logs or the DOS device chain (`CON`, `AUX`, `PRN`). A dump is real when it contains book strings at the file offsets already known, as `page.bin01511` did (`ASYM_TpID`, `Page#`, matching `9e954d01.tbk` at `0xB8B51`).

Do not mix these commands into a WDW session. The command card, the screen, and the 15 October book-open read are Appendix G of the reference manual. The assembly frame that card depends on is Appendix I. This chapter is the claim list. It is not a second copy of those cards.

Standing orders that stay true, and that the short operational notes had in one place: `BC *` before the next breakpoint, because a live `GDI!POLYGON` breakpoint freezes SoftICE when the page scrolls. Segment numbers in `MAP` output are hex. Do not search all of memory. A wide search for `80 65` GPFed the VM. `BPX BAS(82):0376` is not an address. `MAP`, then the live selector, then the offset.

7.2 WDW

On the Watcom debugger:

- Create the breakpoint first, then set `resume on`.
- Qualify with `@`, not `!`.
- Log with `log <file>`.
- There is no trace that prints every call. The three logs in `tbwp0026` are named-export hits: 11 in W16, 53 in UTL, 173 in BAS.

Those three logs are described in chapter 12. Appendix H is the Watcom screen and the command words. `!` still does not work there.

On 2026-10-16 every `TB80RTM` export was broken in the Functions window before the run, and broken again after one single-step once the program was running. The program ran from start to end. Nothing fired either time. At the end the Assembly window was `0000:0000` and `????`. The miss is not "the breaks were set too early." WDW was not attaching those names to the code that executed. The 15 October hits were unpacked W16, UTL, and BAS, with `break <symbol>` and `@`. That has not been shown to work for this RTM window.

GPF notes from the SoftICE sessions still apply in spirit: do not step into an unloaded segment, and do not single-step a far call into `KERNEL` unless that segment is present. The CMP expand host died in `TB80W16` at `001c:00000015` (`9A 9C 06 57 2A`, segment not present). That fault is parked. It is not the book-open path.

7.3 Unicorn

`unfold.py` enters 16-bit protected mode, calls `BootApp`, and dumps each segment `LoadAppSeg` materializes. Pass 01 filled 3 of 139 RTM segments. Pass 02 filled 138, after the call stub was flushed from the Unicorn cache so every call did not replay segment 2. Passes 03 through 07 bind imports, including 461 ToolBook ordinals to real DLL segments. 339 imports stay Windows stubs. The host still dies at `FFFF:0158` and `0080:0098` before a DOS open of the book.

Thought (WV-008). A real Win16 return is what would get that host further. The three hosts (the RTM experiment, `newl`, `neww`) are not one program and do not share a Windows table (WV-035).

The path that does open a page is the pipeline host, which parses the file instead of running RTM. On `9e954d01.tbk` that open is at `0x9EC`, 129 pages. `render.html` is written only when the debug-render flag is set (WV-034).

7.4 Password cipher

`TBCBT` segment 7 (Ghidra `Code7`, base `1030`) holds the four cipher exports. A SoftICE module list that showed segments `01`, `0E`, `02`, `0B`, `08`, `0D`, `04`, `03` had not loaded segment 7. The dialog is an ordinary Windows dialog. The cipher is not in the dialog procedure.

After a failed password, the typed string and the constants `0xA01` and `0x300` remain in memory. Searching for those constants finds the call site even while segment 7 is unloaded. The three addition schemes, the with-password file delta, and the fact that the stand-in key does not decrypt it are chapter 16. Section 16.5 is the 16 October author session: `sssssssssss.tbk` became password-protected and `TBCBT` was not in the module list. The NIDA `.sbk` passwords are chapter 16.6. They are not required to read the records.

7.5 Ghidra

Two NE loader bugs were fixed for this work: the entry-table segment byte was treated as signed, and `ne_cbenttab` was ignored. `TB80BAS` is the image with entry segment bytes at or above 128. The unpacker does not rewrite those bytes. The Ghidra patch does. An honest BAS image loads in the patched Ghidra 12.1.4 build. The procedure for finishing the listing and renaming over time is Appendix K of the reference manual.

Force-disassemble scripts recover functions the loader never reached. One walks undefined `C8` (`ENTER`) bytes. The next accepts a real `RET` / `RETF` before treating the following byte as code, and it only creates functions at entry-like symbols. Run `C8` first, then the prologue pass. They do not name anything, and they do not touch code that is already defined.

`SENDMESSAGE` (321 hits in one RTM listing) and `LSTRLEN` (282 hits) have mangled arguments. That is a decompiler problem, not a missing function. It wants a fresh decompile, not another reading of the same HTML.

7.6 Bochs

The debugger is outside the guest. A breakpoint does not need SoftICE, and it does not need a program inside Windows. The cost is speed.

The command card, the start line, and the stops from the 20 October lesson open are Appendix L. The selectors from that boot are `maps/bochs-20261020.md`. They are dead after the next reboot. Rebuild them before setting `vb`.

What that session watched is chapter 4.6. This section does not repeat it.

8. Naming

Claims

Id	Grade	Statement	Source
N-001	Known	Eight modules were counted for pattern coverage: 10,991 unique functions, 1,918 nameable by pattern, 17.5 percent.	`toolbook_naming_progress.md`, 2026-10-11
N-002	Known	Confirmed renames, after the later digs, are 112: BAS 20, then BMP, RTM, CMP, UTL, W16, LNL, MM, RCR, FLT, NET, JPG, and VBX.	`naming_summary.md` through 101, then JPG +1 and VBX +6
N-003	Known	A pattern name is not a confirmed name. The nameable lists are candidates. The rename scripts are the ones that were applied.	`TBdecompile/rename/`
N-004	Known	Coverage by module, pattern only: RTM 32.7 percent, UTL 41.1, BMP 26, BAS 17.4, W16 17.3, LNL 12.2, CMP 5.6.	progress table

8.1 What the numbers mean

“Nameable” means the decompiler text matches a pattern: a GDI call, `GLOBALALLOC`, `SENDMESSAGE`, a string move. “Confirmed” means a person or a script renamed that one function because the body matches the name. The gap between 1,918 and 112 is mostly unreviewed candidates, not unknown programs.

The 30 percent target in the October 11 note was a counting goal, not a research goal. Naming another thousand `StrCpy` wrappers will not find the record reader.

8.2 Confirmed painters and neighbors

These are the names later chapters use. Addresses are Ghidra `segment:offset` on the image those scripts were run against. Selectors move. Offsets inside the NE segment do not, once the segment is identified.

Name	Address	Module
`PaintPolygon`	`1408:0376`	BAS
`PaintPolyline`	`13e8:00E6`	BAS
`PaintRectangle`	`1020:03E4`	BAS
`PaintArc`	`10A0:0558`	BAS
`PaintEllipse`	`1230:2F1C`	BAS
`PaintRoundRect`	`1230:2278`	BAS
`ScaleTwipsToScreen`	`1008:01DD`	BAS
`SetProperty`	`12D0:03EC`	BAS
`SetPointArray`	`1408:00D8`	BAS
`SetupPaintRect`	`12D0:086C`	BAS
`SetScriptProperty`	`1008:03F6`	BAS
`Lz77Expand`	`1000:0031`	BMP
`CreateDibBitmap`, `BitBltBmp`, `SelectPaletteBmp`	BMP rename script	BMP
`LoadCompilerResource`, `ExpandTextCore`	CMP rename script	CMP
`PaintPageRect`	`1398:05C6`	RTM

Ellipse, round-rect, and a second arc painter were added in the same BAS pass. They are real GDI wrappers. They are not evidence that every shape in a lesson is stored as that GDI call’s arguments. Chapter 5 is the rule for storage.

8.3 Scripts

Java and Python rename scripts live in `TBdecompile/rename/`. Copies of the nameable TSV files live under `TBWorldview/Modules/tb80*_knownnames/`. The scripts rename only when the old `FUN_` name is still present. They do not invent names for functions that do not match the body.

Ghidra `CodeN` is not the NE segment number. Use `TBdecompile/maps/` before typing a segment into a debugger.

8.4 What is still thin

`TB80RCR` exports are only partly named. The `TB80UTL` classification strings are known. They are not yet tied to the kind bytes inside `14 00 00` chunks. CMP is 5.6 percent pattern-nameable because most of it is the compiler, not a wrapper around a Windows API. A name on a CMP function is useful when it is `COMPILEREXPANDTEXT`. A name on a CMP function that only moves a token is not.

9. The modern stack

Claims

Id	Grade	Statement	Source
M-001	Known	The clean-room reader is not the engine. It does not score quizzes, place every bitmap, or run the NE images.	WV-005
M-002	Known	Foolbook 1.0.2 opens `.tbk`, `.sbk`, `.fbk`, and `.fsk`. A polygon becomes the point array `Polygon` is given. Twips convert with `(coord * dpi + 720) / 1440`. Each object is placed from its twip bounds. Twip `0, 0` is the window origin.	Foolbook `v1.0.2`
M-003	Known	Foolcourse is the process that serves a book. Foolbook is a local window. If Foolbook is started by a course, it pulls the book and skips the splash.	release notes, 2026-10-13

Id	Grade	Statement	Source
M-004	Known	The dependency order is Foolversity, then Foolcourse, then Foolbook, then `openscript_analysis`. Foollege is the layer under Foolversity that holds courses. It is not a running server in the current release.	hierarchy note
M-005	Known	TBpipeline 1.0.0r4 parses `9e954d01.tbk` (129 pages) without running RTM. A known bug is an 18-versus-23 render count.	pipeline release note
M-006	Thought	A reader that prefers explicit points, then TBX Vertices, then bounds, will look like the lesson often enough to study it. It will not be a substitute for the Win98 runtime until the reader in chapter 4 has a name and the `0x4097` slot is located in the page blob. The property's name is already known.	project philosophy

9.1 Foolbook

The engine parses the 8.1 container. It does not load `TB80BAS`. Polygons use the kind `0400` layout (count at `+0x1A`, pairs at `+0x22`). Rectangles use the bounds-only tail (`01 00 00 04`). On Logic Families that is on the order of 165 shapes from the early polygon-and-rect pass, and 961 rectangles from the later marker scan. Both counts can be true. They are different patterns.

Foolbook 1.0.2 places those shapes before the viewer sees them. The conversion is the one in chapter 5: `(coord * dpi + 720) / 1440`. At 96 dpi that is the `/15` case, with the 720 added so the division rounds. The placed polygon is a pixel point list. The placed rectangle is `x, y, w, h` in the window. Twip `0, 0` is the origin. The tag is `v1.0.2`. Foolcourse `v1.0.0r3` and Foolversity `v1.0.0r3` depend on that tag.

System books: core `.fsk` first, user `.fsk` after, as mods. `.sbk` is accepted. It is not required for the v1 player.

The window is native. A direct start opens `using_foolbook.fbk` and a splash that asks whether to resume the last page. A start from Foolcourse, or a start on a named file, skips the splash and uses the last page the course recorded. The splash text is a joke (`2+2=5` / "For greater values of 2."). It is not part of the book format.

OpenScript in this player is the subset in chapter 6. ActiveX, Python, and C++ as replacement engines are a stated direction. They are not implemented.

The book API and the object layer are separate modules in Foolbook (`engine/cdb.py`, `engine/obj.py`). A property get locks the `0x76` slot, runs the behavior, and unlocks. File open reaches the object file through the unnamed helpers, not by a direct call. That is chapter 20. It does not load `TB80BAS`, and it is not the reader.

9.2 Foolcourse and Foolversity

Foolcourse is the only server in the current release. Foolbook does not listen. The earlier idea that Foolbook itself served on port 1337 was withdrawn: a local viewer should not be the thing that stays up after the tab closes.

Foolversity writes students, schools, and curricula. Foolcourse reads them. The hierarchy, as stated, is: a Foollege sits under Foolversity and has Foolcourses; a Foolcourse has Foolbooks; a Foolbook can run OpenScript. Foollege is not the current server. A later Foolcollege was discussed as the place that would hold several courses for several students. That program does not exist yet.

Install is a virtual environment and `pip install` of the Foolversity git URL. A system Python on Debian will refuse the install. That is PEP 668, not a missing package. If a dependency has not changed, a reinstall should not download it again unless a fresh-start flag is set. That rule was written after several reinstalls pulled `openscript_analysis` for no content change.

9.3 TBpipeline and the unpacker

TBpipeline's useful stage is the file parser: load order recorded from a SoftICE log, then a TBK open, then a page walk, then HTML. The Unicorn RTM stage still stops before the book is opened, so a render from this package is a parse, not a playback. Curves, including an attempt at the P420 flame, are emitted as separate files. They are not placed on the page (WV-004's falsifier still stands for placement).

`toolbookunpacker` produces an honest NE image. It clears the Optloader self-load flag. It does not patch the entry table for Ghidra. That patch lives beside the unpacker as an optional post-step. Only BAS, on the current set, has entry segment bytes at or above 128.

9.4 What "getting NIDA working" means

Three different finishes are easy to mix up.

Finish	Done when
A page can be read	Field text, jumps, and the decoded shapes for that page match the Win98 screen closely enough to study the lesson.
A lesson can be walked	Next, Back, menus, and quiz flow follow the handlers, including the jumps into `nida81.tbk`.
A lesson can be run	The system books load, `initializebase` / `initializerecs` / `initializetest` do what the DLL exports imply, pictures land on their widgets, and the unnamed reader is no longer unnamed because the clean-room code does the same thing for the same bytes.

The first is partly done for Logic Families. The second is partly done for the menu and the quiz ids that have been transcribed. The third is not done. Authorship and a modern `TB80CMP` are side work. They help the next person. They do not open P1040.

10. Open, retired, and superseded

10.1 High

These block a faithful page.

Gap	Grade	Why it is still open	Next step that would close it
Runtime record reader	Known as the read in chapter 4.6: `1178:08B2` → `1188:04DA` → `1188:08D6` → `1160:02CC`.	The bytes are in memory before message `0xE`. Still open: the dispatcher for handle `0x0C36`, and which fields of the 412-byte body are the page.	One Bochs stop at the return of `1180:03C8` for that handle. Appendix L. Do not break ordinal 51.
Picture id	Known for a button and the directory (G-012).	The id is the fourth word. The button stores it plus one. A background picture is the block at `+0x37` when that block is present.	Which of the non-bitmap huge entries are palettes.
Property expansion	Known that `FUN_12c8_0ef8` copies four words from a structure that is not the disk record.	Offsets `+0x3E` and friends do not match a `14 00 00` header.	Only useful on the author path. For runtime, fold this into the reader break. Do not keep it as a separate `CDBSENDCREATE` hunt.
Tbload vtable	Thought. `FUN_00406a70` calls vtable+4 after the marker search.	The 32-bit search is found. The callee is not.	One break at `00406aa1` with the book path in a register. Low value until the reader is the goal again.
Polygon type table	Known for the specimen.	Class `0xF` and class `0x10` both call `POLYGON`. This book uses `0x10`, count at `+0x30`, points at `+0x32`.	Class `0x14` curves, and the eight class `0x19` records in `nida81.tbk`.

10.2 Medium

Gap	Grade	Note	
P420 stroke breaks	Thought	Three 96-point groups in TBX. One tangled dump from `+0xA13`. Look for length prefixes between runs before inventing a new curve format.	
Arc on disk	Known live, unknown on disk	Type `0x11`, four pairs, fill at `+0x18`. Angles not located. No `GDI!PIE`.	
Stroke color and pen width	Stroke known (G-011). Width is the pen table	Fill is `+0x1B` on a draw object and `+0x5B` on a background. Styles 1 through 9 are none, dot, dash, solid, then inside-frame widths 2, 3, 4, 6, and 8.	The table is `DS:0x1708` in the unpacked DLL. Not re-checked on a painted line.
Palette on an 8 bpp BMP	Known that segment `1150` builds a palette	Not known which palette a given carved BMP must use.	
`DRAWTEXT` beyond three offsets	Open	`param_5+0x4A`, `+0x4C`, `+0x50` are the paint path. The disk handle is word 1 at `+0x24` (O-009).	
Per-pin index `+0x52`	Known the field exists	Not applied. Shared point bytes stack.	
Background, parent, z-order	Background known on disk	Page `+0x47` names the class 4 record, 131/131 (O-011). Parent is the near pointer at `+0x04`. Z-order is the list at `+0x2C`, not a byte.	The on-disk bytes of that list header were not dumped.
`initializebase` and `toolbook.lst` order	Thought	Names are in the lesson. The call log is not.	
OpenScript keyword lock	Thought	Instructor 8 compile, then expand. Not run.	
`SENDMESSAGE` / `LSTRLEN` arguments	Known mangled	Re-decompile. Not a new algorithm.	

10.3 Retired

Idea	Grade	Why it stays closed
On-disk `14 00 00 01` becomes `14 00 00 11` in memory	Not true	Both values exist on disk. A later live object used `14 00 00 02`. WV-010.
`FUN_1028_2680` is the reader	Not true	Filter registrar.
`FUN_1008_2d31` is the reader	Not true	Math.
RTM contains the reader	Not true	Paint handoff only.
`CDBCHUNK*` is the reader	Not true	Text separators.
`GDI!PIE` draws pies	Not true	No import.
The reader must contain the literal bytes `14 00 00`	Thought, not a requirement	Length-prefixed walks would not. Do not close a function for lacking the literal, and do not keep scanning for the literal as the only method.
HTML listing absence	Not evidence	`??` runs and forced-disassembly passes exist.
`TB80BMP` resolves a stored bitmap id	Not true	The id is a type directory in `TB80BAS` segment `1028` (G-010). BMP's palette notify can still run.
`TB80FLT` reads a bitmap out of a book	Not true	It is the import filter. A picture already in the book does not go through it.

10.4 Superseded notes

Read these for the function bodies. Do not read them for the runtime path.

Note	What it got wrong
`tbworkingpapers/tbwp0002/bas-create-path.md`	Put `CDBSENDCREATE` on book open.
`tbworkingpapers/tbwp0008/cdbsendcreate-sites.md`	Same chain, more callers. The callers are author-side.
`TBdecompile/trail/20261015/00-how-toolbook-operates.md` §2	Same chain, written as “the create path that has been identified.”
`TBdecompile/trail/20261015/01-theories-of-operation.md` §1	Still calls the reader “the caller of `CDBSENDCREATE`.”
`TBdecompile/trail/20261014/02-still-open.md`	Same wording. The rest of that file’s open list is still the open list.

Note	What it got wrong
`TBWorldview/ToolBook_Architecture.md` book-open flowchart	Draws `CDBSENDCREATE` once per widget.

The correction, in one paragraph: Instructor calls `CDBSENDCREATE` when it creates or rewrites an object. A lesson open does not. The function that turns a `14 00 00` chunk into a live object during that open has not been named. It does not call ordinal 51. `FUN_12c8_0ef8` is on the author side of the same line.

10.5 Side quests that are real and not the reader

Kept because they will matter after the reader has a name.

- Object pool: `FUN_1478_014a` allocates near objects. `FUN_1478_00a0` is the pool.
- RTM dialog chrome: `GETDLGITEM` id `0x65` (55 hits), `GETWINDOWLONG` index 10 (196 hits). `FUN_1068_0098` is `GLOBALALLOC` with an out-of-memory retry.
- Author mode exports on BAS, and `GETAUTHORLOCALIZEDDLL` on RTM. The author binaries themselves are not in the archive.
- Password cipher in TBCBT segment 7. Constants `0xA01` and `0x300` after a failed password.
- CMP expand, parked at the W16 segment-not-present fault. Not on the NIDA open path.

10.6 Order of work

1. One lesson open under WDW. Break `TB80BAS@CDBFILEOPEN`, resume, then break the next BAS export that runs before the first paint. Log it.
2. When an object exists, dump `+0x0F` and the page offset it came from. That pair names the reader, whatever the export happens to be called.
3. Only then place bitmaps, split the P420 strokes, and apply `+0x52`.
4. Do not schedule another full-module naming pass, another Unicorn boot, or another break on ordinal 51 in service of step 1.

10.7 The loose sheets are not a second record

Chapter 18 names every markdown sheet under `tbworkingpapers/` and the section of this directory that replaced it. A sheet can still describe a real function and still be wrong about when that function runs. Leaving the sheet on disk keeps the trail. It does not keep the old claim alive. The printable form of the claim is the PDF next to these chapters.

11. Keepers

Addresses in this chapter are from the 2026-10-15 Windows 98 boots, DGROUP `347F`. Match a later boot on the `enter` size and the next two or three instructions, not on the selector. Decompile line numbers are from `TB80BAS.DLL.c` and do not move when the selector does.

`CDBSENDCREATE` is not in this list. It is author-only. See chapter 4.

11.1 File open and close

Function	Live (one boot)	Enter	Decompile line	Role
`CDBFILEOPEN`	`4C7F:017A`	`000A`	27592	Book-open export. Ordinal 8.
`CDBFILECLOSE`	`4C7F:0948`	`0002`	27965	Close. Same caller cluster as open.
`CDBFILEPATH`	`4C7F:09E0`	`0002`	28002	Path query.

Function	Live (one boot)	Enter	Decompile line	Role
CDBFILESTATUS	3ACF:0000	0004	30210	Status.
CDBQUERYMULTISAVE	4C7F:0B90	0018	28108	Multi-save query.
CDBMULTISAVE	4C7F:0E24	000C	28272	Multi-save.
OBJFILEOPEN	3147:177C	025A	76162	Large frame. Buffered read. Downstream of the reader.
OBJFILECLOSE	3147:10EA	0002	75887	Object-file close.
OBJGETFILE	3147:0094			Object file access. Seen on the page path.
OBJGETHDRLONG	3147:0242			Header long. A 4-byte read here was an internal pointer, not a page header.

CDBFILEOPEN callers seen: 322F:3252 (jump), 3D6F:0416 / 0F79 / 0E8E, and the 3247 spine. OBJFILEOPEN callers include 3D6F, 3247, and OBJGETFILE.

11.2 Object lifetime

Function	Live	Enter	Decompile line
OBJFREE	3147:0CCC		75619
OBJFREENEAR	3147:0D70		75662
OBJUNLOCK		004A	call site, not a definition
OBJREALLOCNEAR	3147:08D6	0018	
CDBCLEARBOOKREFS	4C7F:10A6	00B8	28451
CDBINIT		00C8	30304

HEAP* / TBHEAP* do not hand out the 0x76-byte object. That sentence, and the sentence that TBHEAPCLEARCRITSECT has no definition, are not true. TBHEAPCLEARCRITSECT is decompile line 93984. The object array is GLOBALALLOC of count * 0x76 in FUN_1278_3be6. Chapter 19 is the heap, the value stack, and RESMGR*. TBHEAPCLEARCRITSECT was a stop on the open path. It was not the last stop before the window. VALUEFREE was.

11.3 Page, instance, and the value bus

These are closer to the unnamed reader than ordinal 51. On the 2026-10-15 captures, CDBSENDCREATE was listed in the Functions window and did not appear in the Calls stacks. CDBCREATEPAGEREF and CDBSETVALUEEX did.

Function	Live	Enter	Decompile line	Note
CDBCLASSTABLEGETBEHAVIOR	44F7:121A	0008	125154	Splash gate. Unbreaking it let the splash run.
CDBCREATEINSTANCE	480F:0000	0002	24974	Instance constructor.
CDBCREATEPAGEREF	483F:03B2		25913	Page reference. Next break.
CDBVIEWNONE	483F:02F8	0002	25877	View state.
CDBVIEWPAGE			25726	Ordinal 15.
CDBVIEWBKGND			25825	Ordinal 18.
CDBSETVALUEEX	3397:147C			Extended property set. Next break.
CDBGETVALUE	3397:0D54		38475	Property bus.
CDBGETVALUEEX			38488	Extended get.
CDBPARENTVALUE	3397:07B2			Parent value.
CDBDEREFVALUE	3397:0400			Deref.
CDBLOCKVALUE	3397:02CA	0002	37826	Shifts 0B[bp]. Not a record parser.
CDBCLIENTSTATUS	480F:04C6			Client status.
CDBREGISTERINSTWINDOW	480F:0522			Instance window.
CDBDISPLAYSTATE	4877:0000			Display state.
CDBSYNCSCREENS	4877:0276			End of the author create path. Seeing it does not mean ordinal 51 fired.
CDBSENDSELECT	44CF:009E			Select. Not create.

Value constructors in segment `4267` fire often and are not the stream walker:

Function	Live	Enter
CDBVALUENEWBOOKREF	4267:0000	0004
CDBVALUEBOOKFROMSTRING	4267:03F4	0018
CDBVALUELAYERFROMSTRBOOK	4267:05F8	004C
CDBVALUERESOURCEFROMSTRING	4267:08B2	004A
CDBVALUEGRAPHICFROMSTRING	4267:0AA0	0074
VALUENEWBOOK	4267:0D90	0006
VALUENEWRESOURCE	4267:0EFC	000E
VALUENEWGRAPHIC	4267:1494	0008
CDBVALUENEWGRAPHICREF	4267:02D4	
CDBVALUENEWLAYERREF	4267:00C2	

`VALUENEWLONG` builds a long. It is not a record parser.

11.4 Chunk walkers

These walk characters: comma, CR, LF, tab, space, quote. A decompile that shows `ISDBCSLEADBYTE` and a comma test is this family.

Function	Live	Enter	Decompile line	Callers
CDBCHUNKEND	0D37:0000	0004	138456	3187:0210, self from FIND
CDBCHUNKFIND	0D37:0146	0008	138575	4857:0C68, 3187:0210, 322F:0577
CDBCHUNKLAST	0D37:0334	0010	138676	4857:090B, 3187:0210

11.5 Paint and resource binding

Function	Live	Decompile line	Note
CDBPAINTCLIENT	389F:0506	13056	Ordinal 26. RTM `WM_PAINT` ends here.
IMBITBLT		13277	BitBlt path.
IMDRAWLEVELS		13353	Draw levels. `IMDRAW` as a bare name is not a definition.
IMBLACKOUTBMP		13995	
IMINVERTBMP		14021	
IMSHOWBMP		14047	
RESMGREXTRACT		21513	Resource manager.
RESMGREXTRACTGDI	4D27:1730	21523	Extract for GDI.
GETDIBCOLORCOUNT	35D7:053A		Enter `0006`.
GETDIBSIZE	35D7:05DA	not in the BAS or BMP listing	Enter `0008`.
HUGEGETDATA	33BF:0D6C		Enter `0018`. Large fetch.
COLORHLSTORGB	44B7:0B84	155991	Menu color. Enter `0034`. Not the page painter.

Painter offsets that do not move inside the segment are in chapter 8. `PaintPolygon` is BAS segment `0x82` offset `0x376`.

11.6 Incidental on open

Seen, and not the reader: `CDBREGISTEROLE2WINDOW` (`3767:0000`), `CDBREGISTERVBXFORM` (`4437:0000`), `VBXNEWVIEW` (`4437:0216`), `VALUEARRAYINFO` (`4357:02F2`), `VALUEARRAYSET` (`4357:0932`), `VALUEARRAYGET` (`4357:05D8`), `HEAPVALUETOPOINTER` (`450F:0F28`), `HEAPVALUETOFRAME` (`450F:0F4E`), `HEAPENTERCITSECT` (`450F:0DCE`), `HEAPLEAVECRITSECT` (`450F:0DF6`). What the heap stops are is chapter 19.

`?SENDPROC@TBXBOBJ@@` was seen at `29AF:0000` (`cmp ax, 3014`). The decompile does not contain `SENDPROC@TBXBOBJ` as that name. Related symbols are `_SETPROC_TBXBOBJ` at line 2175, `_SENDPROC_TBXLAYER`, `_SENDPROC_TBXBASE`, and `_SENDPROC_TBXUSER`.

12. The book-open spine

12.1 The frames that repeat

On the 2026-10-15 book-open and page walk, these frames were in nearly every WDW Calls window. Selectors are that boot only.

Selector:offset	Note
`3497:03EF`	Early frame.
`3497:4A6E`	The planned step point.
`3197:0379`	
`3167:0CB1`	Near call.
`3167:14BC`	Far call.
`3247:02BF`	Near.
`3247:05B7`	Near.
`3247:0B7A`	Far.

Everything else in those captures hangs off this spine. The full selector-to-decompile table for that boot is `maps/wdw-to-ghidra.md` (485 lines). It is a lookup, not a finding. Do not treat a selector from that file as valid on the next boot.

12.2 What the three export traces contained

WDW has no full call trace. `break <symbol> resume on` plus `log <file>` logs named exports and continues. `resume on` is a property of a breakpoint that already exists. Module qualification is `TB80BAS@OBJFILEOPEN`. SoftICE `!` does not work in WDW.

Trace	Shots	What fired	What did not
W16	11	`GETPLATFORMREF`, `IS32BITENV`, `GETRUNTIMELOCALIZEDDLL`, `ACXINITIALIZETOOLBOOKSUPPORT`, `ACXOLEINITIALIZE`, `ASYM_STRINGFROMCLSID`, `SETTHUNKINGLAYERSERRORFUNC`, `LOADLIBRARYEX32W`, `GETPROCADDRESS32W`	Book open. Record walk.
UTL	53	`GETCHARINFO`, `GETSTRMGRSTRING`, task collections, `PLUGINSVCINIT`, `LOADRESSTR`, `CXTHELPINIT`, `GETTBKINIPROFILEINT` / `STRING`, `INITMETRICS`, `SETCONVERSIONDC`, `CXTTOPXT`, `GHEAPCREATE`, `GHEAPALLOC`, `SYMLOOKUPNUMBER`	Record walk. The type-name table lives here and was not the thing being stepped.
BAS	173	`VALUETOSTRING`, `VALUENEWSTACK`, the `VALUENEW*` family, `OBJFILEOPEN` (`enter 025A`), `STACKGETSIZE`, `STACKPUSH`, `CDBCHUNKLAST`, `CDBCHUNKFIND`, `CDBCHUNKEND`, `RESMGR*`	A compare of a far pointer to `0x14`. Ordinal 51.

What those stack, heap, and `RESMGR*` hits do is chapter 19. They are still not the reader.

`OBJFILEOPEN` in that BAS trace is the file-open path. The buffered reader is `FUN_1160_03de`, reached through `FUN_1258_00a8`. When the buffer satisfies the request, the raw DOS path at offset `04F3` is skipped. Live 4-byte reads returned an internal pointer (`FC B5 BD B0` in one dump), not a page header and not `03 4A 42 4F`.

12.3 Where to break next

1. `TB80BAS@CDBFILEOPEN` on a lesson open. Resume.
2. Then `CDBCREATEPAGEREF` or `CDBSETVALUEEX` (export names, not the selectors above).
3. When an object exists, dump `+0x0F` and the page offset it came from. That pair names the reader.
4. Do not break ordinal 51. Do not schedule another pass over the same W16 and UTL exports in order to find the reader.

The reader is still unnamed. The working model is in chapter 4: a length-prefixed walk in BAS after `CDBFILEOPEN` and before the first paint, with no literal `14 00 00` compare required.

12.4 Debugger facts that keep wasting a session

SoftICE field work closed 11 October 2026 (`SI405w9x` plus ICEDUMP 6.021). Ordinal breakpoints survive a reboot. Selectors do not. `TB80BAS.Ordinal_8` is `CDBFILEOPEN` on every boot.

`/DUMP` writes the SoftICE console unless the dump experts are set. Several early `page.bin` and `rtm.bin` files are command logs or the DOS device chain (`CON`, `AUX`, `PRN`). A dump is real when it contains book strings at offsets already known, as `page.bin01511` did (`ASYM_TpID`, `Page#`, matching `9e954d01.tbk` at `0xB8B51`).

Symbol maps are `NE-segment:offset` names (`TB80BAS.map`, 3799 entries; `TB80RTM.map`, 2669). SoftICE and WDW do not load a `.map` as a module.

The CMP expand host died in `TB80W16` at `001c:00000015` (`9A 9C 06 57 2A`, segment not present). That fault is parked. It is not the book-open path.

The Unicorn host has not opened a book. Passes 01–07 fill segments and bind 461 ToolBook ordinals, then die at `FFFF:0158` and `0080:0098`. Pass 06 exits after `TB80.INI`. The three hosts (the RTM experiment, `newl`, `neww`) are not one program.

13. Specimen pages

Every number in this chapter is from `9e954d01.tbk` (Logic Families) or from its ToolBook 11.5 Save To XML, unless a path says otherwise. Screen pixels are stored twips / 15 at 96 dpi.

The binary has 129 pages, each a record starting `80 65 00 05 00 02 02`. The export has 131 `Page.xml` files. P440 and P2920 have no `80 65` record of their own. All 129 binary names match a TBX page. P440's sentence and the house shapes that sit on its TBX bounds are inside the P430 record. See 13.5. P2920 has not been searched the same way.

TBX folder names (`Page880`) are export ids, not the runtime page name (`P1040`). IdNumber 880 is page `P1040`.

13.1 P1040 — two DIP packages, bounds only

File offset `0x19C03A` (1687098). Page blob 7764 bytes. Background id 2. Next goes to `P1050`. Back goes to `P1001`.

The 8.1 file does not store the 11-point pin tips that the XML writes. Each pin is four twip words, a zero dword, fill `0xC0C0C0`, then marker `01 00 00 04`. The tip is expanded at paint, and the export writes that expansion as Vertices. The book has 961 copies of the `01 00 00 04` marker. P1040 uses 28 of them for pins and 2 for bodies.

7400, right package

Body at blob 0x09F4: bounds 4875, 3555, 7830, 4605, screen about 325,237–522,307, fill 64,64,64 (0x404040). Label field text 7400.

Bottom pins, top 4605, bottom 4860:

Blob offset	Bounds (twips)	Screen about
0x03E4	5055–5205	337,307–347,324
0x044C	5460–5610	364,307–374,324
0x04B4	5880–6030	392,307–402,324
0x051C	6315–6465	421,307–431,324
0x0584	6705–6855	447,307–457,324
0x05EC	7125–7275	475,307–485,324
0x0654	7560–7710	504,307–514,324

Top pins, top 3300, bottom 3555:

Blob offset	Bounds (twips)	Screen about
0x071C	5055–5205	337,220–347,237
0x0784	5460–5610	364,220–374,237
0x07EC	5880–6030	392,220–402,237
0x0854	6315–6465	421,220–431,237
0x08BC	6705–6855	447,220–457,237
0x0924	7110–7260	474,220–484,237
0x098C	7530–7680	502,220–512,237

The first pin's bounds 5055, 4605, 5205, 4860 are the start of the TBX Vertices list. That is the expansion direction: bounds in the 8.1 file, point list in the XML.

5400, left package

Body at blob 0x117C: bounds 1500, 3555, 4455, 4605, screen about 100,237–297,307, same fill 64,64,64. Label 5400.

Bottom pins, y 4605–4860: 0x0B6C 1680–1830, 0x0BD4 2085–2235, 0x0C3C 2505–2655, 0x0CA4 2940–3090, 0x0D0C 3330–3480, 0x0D74 3750–3900, 0x0DDC 4185–4335.

Top pins, y 3300–3555: 0x0EA4 1680–1830, 0x0F0C 2085–2235, 0x0F74 2505–2655, 0x0FDC 2940–3090, 0x1044 3330–3480, 0x10AC 3735–3885, 0x1114 4155–4305.

TBX group ids: 53 is the 7400 package, 52 is the 5400 package, 58 is the discussion title (gold plate button, graphic "bitmap id 149", plus a white "DISCUSSION" field and a blue drop shadow). The widget still does not name that bitmap inside the 8.1 object. See chapter 5.

Intro field bounds 600, 600, 8805, 2490. Next button 8907, 6210, 9342, 6630. Back button -3, 6210, 432, 6630.

13.2 P430 — kind 04 00 polygons, including a live pin tip

9e954d01.tbk has 96 kind-0400 records on 13 pages. P430 has 22. A search that ignores the kind word produces false positives.

Record layout: fill dword at +4, kind 04 00 at +0x0C, count word at +0x1A, twip pairs at +0x22. The painter is FUN_1408_0376. It copies object +0x30 / +0x32 to DS:30AC and calls GDI!POLYGON.

P430 draws as a chip: silver pins (0xC0C0C0) and a light-gray body (0xE0E0E0). One live 11-point polygon is a pin tip, not the body. Captured screen points ran (218,335) through (218,341), a 10 by 17 notch. The same points are in the

file as twips near page offset 0x9CB: (3147,3862) through (3147,3973).

Pins that share identical point bytes stack until the index at object +0x52 is applied. The clean-room draw does not apply it.

13.5 P440 — no page of its own

The TBX page is Page876, Name="P440", background id 2. The text field says the ICs sentence: classified into logic families by voltage, speed, and how many circuits they can drive. Four houses are irregular polygons. Next and Back are buttons with bitmap ids. The background field named Page# stores the text P1270. That is the background's stored sample, not this page's name.

The 8.1 file has no page named P440. The loader read 129 pages. The sentence is at file offset 834271, inside P430. That page starts at 0xC3312 and its blob is 42512 bytes, so the sentence is 34765 bytes into the blob.

The house shapes are in that same blob. Drawing every shape in it draws the chip and the houses on one window, plus false hits from the scan, including a yellow polygon that covers the page. Keeping only shapes whose bounds overlap the TBX page, and dropping a box larger than the page, leaves the four houses, some chip shapes that share those bounds, and one yellow false polygon. It does not draw the blue background, the sentence, or the button bitmaps. Those are not in the shape extract.

Known. P440 is not a missed 80 65 record, and it is not absent from the file. The sentence and the overlapping shapes are in the P430 record. Thought. The 11.5 export split that one record into a chip page and a house page. That split has not been watched in 11.5. P2920 is the other export page with no 80 65 record, and this search was not repeated for it.

13.3 P420 — the curve that is not yet a flame

Points sit at page +0xA13. Curve's type id in nc81base.sbk is 0x05C7 (file offset 0x1810). That id is not in the P420 page records. A naive walk draws a tangle in screen space around 400–500. The TBX export is three 96-point red IrregularPolygons. There is no Curve name on the page.

Thought. The tangle is strokes concatenated that should be drawn separately. A length prefix or a zero terminator between groups has not been proven. Do not treat a raw dump of +0xA13 as the flame.

13.4 P1170 — the bracket is computed, and the word table is an index

Live polyline: (354,201) (321,201) (321,238) (358,238). Object bounds in twips: left 4801, top 3015, right 5370, bottom 3570. Divided by 15 that is the same rectangle. BAS segment 0x5B computes the four points. They are not stored.

The 27 ascending words at page +0x8C3 are an offset directory into a 14 00 00 83 blob, not a Group. On this page the blob is file 0x1AB651, 1844 bytes, and the word table is file 0x1ABE97. The words begin 0x14, 0x16, 0x40, 0x52, 0x66, 0x7A, 0xC2 and are relative to the blob. Script-like markers inside the blob (buttonClick, "P1180", "P1140") sit at blob +0xA9, +0xB9, +0x17B, +0x18B. After those regions are set aside, the table still points at short structured runs (type bytes, small integers, 0xFFFFFF). A Group is its own object and has its own member list.

13.5 P400 record sizes

P400 is 7046 bytes and 12 records. The early walk, kept because the sizes show what a "page of widgets" actually is:

#	Page offset	Length	Kind	Note
0	`+0x2B`	33	`0001`	Small.
1	`+0x4C`	31	`0000`	Small.
2	`+0x6B`	18	`0D00`	Small.
3	`+0x7D`	1458	`D900`	Content blob: scripts and colors.
4	`+0x62F`	2828	`0021`	Large. Not a polygon.
5	`+0x113B`	894	`0000`	Count word at `+0x1A` is 21, not a kind-`0400` polygon.
6	`+0x14B9`	156	`1600`	Small.

13.6 Lesson jumps that have been read

Page	Handler
P900	TTL → P1001. Other → P2001. Final quiz → P10001. Exit → `nida81` P20001. No Next.
P1001, P2001	Back returns to P900, not to `nida81` P20001.
P300	`selection1` sends `lastbook nida81.tbk` and `nextpage` P310.
P10001	Directions → `nida81` P10020. Exit → `nida81` P20001. Begin → P10210.
P10410	Menu returns to P900.
P1290	Correct choice 5408. Again → P1292. Next → P1300.
P20000 of `nida81.tbk`	`buttonClick` goes to P20001 because the handler says so.

13.7 TBX draw inventory

Tag counts across the `9e954d01` export. These are XML tags, not kind-`0400` records. A page can have hundreds of IrregularPolygons that the 8.1 file stored as bounds.

Kind	Count
IrregularPolygon	654
Line	528
Field	497
Button	382
Group	371
Rectangle	243
AngledLine	66
Arc	62
Ellipse	45
Hotword	5

Pages whose export folder contains at least one IrregularPolygon. Columns are IrregularPolygon, Rectangle, Line, AngledLine, Arc, Ellipse, Group, Field.

TBX folder	Poly	Rect	Line	Angled	Arc	Ell	Group	Field
Page872	14	2	2	0	1	1	4	7
Page873	44	5	0	0	2	2	2	4
Page874	8	2	2	18	0	0	2	2
Page875	96	8	2	1	8	0	8	1
Page876	12	32	108	0	0	0	48	5
Page880	28	2	0	0	2	0	5	5
Page885	28	4	0	0	2	0	4	5
Page887	14	1	0	0	1	0	2	8
Page891	14	1	0	0	1	0	2	8
Page897	14	1	0	0	1	0	2	8
Page900	14	1	0	0	1	0	2	8

TBX folder	Poly	Rect	Line	Angled	Arc	Ell	Group	Field
Page901	42	3	0	0	3	0	3	4
Page909	14	1	0	0	1	0	2	8
Page912	42	3	0	0	3	0	5	7
Page914	28	4	0	0	2	0	5	6
Page915	42	3	0	0	3	0	5	7
Page917	42	3	0	0	3	0	4	8
Page918	42	3	0	0	3	0	4	8
Page919	14	1	0	0	1	0	1	4
Page923	28	2	0	0	2	0	5	5
Page931	3	8	27	0	0	0	13	4
Page932	9	28	81	0	0	1	37	5
Page934	14	1	0	0	1	0	2	2
Page940	6	22	54	0	0	2	24	5
Page941	14	1	2	0	1	0	2	4
Page943	14	1	2	0	1	0	2	4
Page949	2	0	7	0	0	0	0	2
Page961	12	32	108	0	0	0	49	7

Page880 is the P1040 export: 28 polygons, 2 rectangles, 2 arcs. The 28 polygons are the paint-time expansion of the 28 bounds records in section 13.1. Page875 (96 polygons, 8 arcs) is the dense drawing page in the export. It is not, by itself, proof of 96 kind-`0400` records.

13.8 Pictures, stated with the two offsets kept apart

Windows BMPs can be carved. One match against the export paired 67 markers with 72 TBX bitmaps. The widget does not name the bitmap. Compressed pictures go through `TB80BMP` ordinal 3 and have been followed as far as a far call at `0x1FCD`. `Lz77Expand` is BMP `FUN_1000_0031`.

The shared near-start graphic is a 581 by 157, 8 bpp gold plate, not a text banner.

Two notes locate the `nida81` splash and they are not the same byte: file `0x413FAB` (323 by 344, 8 bpp, off-bits 310), and a carved copy taken from file `0x487BE` (27874 bytes). The live `BITMAP` header that was found was width `0x143`, height `0x158`, 8 bpp, stride `0x258`, `bmBits` null. The splash is a DIB. Which property on the page object holds its id is not known.

Lesson picture counts from the early carve: 71 BM and 5 JPEG in the lesson, 76 BM and 12 JPEG in `nida81`. BAS segment `1150` builds the book palette. An 8 bpp BMP needs that palette. Which palette a given carved BMP must use is open.

14. OpenScript bytes

14.1 What is current

Known. Handlers are compressed bytecode. The keyword pool is `TB80CMP` NE resource `0x77` (type 100, id 119). Resource `0x78` (id 120) is the other emit routine. `COMPILERPROPINFOFROMINDEX` locks resource `0x70`. Property names are a different table: BAS resource type `0x2D1`, id `0x2D0` (`bounds` `0x4004`, `idNumber` `0x4015`, `script` `0x4023`, `text` `0x402E`, `vertices` `0x4034`).

An early string dump of resource `0x77` counted 992 strings in 15776 bytes, and of `0x78` counted 980 in 15616 bytes. The decoder that actually produced the 76 Logic Families handlers counts 994 keywords. Use 994. The offset table starts at byte 2 plus count times 4. An earlier loader used count times 2 and was wrong even before the back-reference

bug.

`COMPILERCOMPILETEXT` is CMP ordinal 1. `COMPILEREXPANDTEXT` is ordinal 3. `CFUNEXECUTE` is 8. `CFUNEVALUATE` is 9.

14.2 Encoding

From `FUN_1040_0970`. This is the encoding the handlers decode with.

Range	Meaning	
`0x00`–`0x7F`	Literal character.	
`0x80`–`0xBF`	Keyword. Index is `byte & 0xF`, or `((byte & 0xF) << 8) \	next` when bit 4 is set. A space is appended when bit 5 is clear.
`0xC0`–`0xFF`	Back-reference. Copy from the compressed input stream, not from the output.	
`0xE0`–`0xEF`	CRLF plus spaces.	
`0xF0`–`0xFE`	Spaces.	
`0xFF`	Repeat.	

The `0xC0`–`0xFF` range is also where the whitespace classes sit. The expander classifies the byte before it treats it as a generic back-reference. Do not implement "every byte above `0xBF` is a back-reference" and also implement the whitespace rows as a second pass over the same byte without the classification `FUN_1040_0970` uses.

Back-reference, as fixed on 2026-10-10:

```
if bit 4 is clear:
    length = (byte & 0xF) + 3
    src = (pos + 1) + (-4 - next_byte)
else:
    length = (byte & 0xF) + 4
    src = (pos + 1) + (-1 - word_offset)
copy `length` bytes from input[src]
```

A windowed decoder that only looks inside the slice under test will mark a back-reference that lands before the slice. On `tb80r.sbk`, reading those from the whole book dropped the `?` marks from 14256 to 66. The 66 that remain are question marks in the script.

The NIDA navigation decoder stops at the first standalone `end`. Trailing garbage is dropped. That is a corpus tool, not a claim that the expander itself stops there.

14.3 The superseded mismatch

`tbworkingpapers/tbwp0001/openscript_notes.md` says, in its first conclusion, that the decoder emits "italic", "third", "fourth", and "systime" where the script should say something else, and that the bytecode might not be OpenScript. That conclusion is superseded by the same file's later sections, and by this chapter.

Two bugs produced it. The keyword offset table was read as count times 2. Back-references copied from the output stream. After both fixes the same bytes are:

```
system lastbook
  lastbook = name of this book
  system nextpage
  nextpage = name of this page
  go to page "P20001" of book "nida81.tbk"
end
```

Other handlers that are real source: `send next`, `send previous`, `go to page "P400"`, `go to page "P310" of book "nida81.tbk"`.

Do not reopen "the keyword table is the wrong resource" from the early half of that note. Keywords live in `TB80CMP`. `TB80BAS` has no `handle` or `buttonClick` strings. `TB80RTM` has a small table at file `0xAACAD` (`handle`, `get`, `set`, `notifybefore`, `notifyafter`, `true`, `false`, `to`) and not the compiler keyword pool.

14.4 Corpus

Book	Handlers	How they are stored
`9e954d01.tbk`	76, on 39 pages	Plain `to handle` prefix, compressed body. An intermediate extract that required a plain `END` found 22. 76 is the count after the input-stream fix.
`nida81.tbk`	100	All `buttonClick`. Same encoding.
`tb80r.sbk`	33	The `to handle` / `to get` prefix is compressed too. A scan that keeps windows whose output starts with those words is how they were found. More `0xFF` and `0xE0` than the lesson books.
`nc81base.sbk`, `nc81recs.sbk`, `nc81test.sbk`, `nc81trnr.sbk`	0 decoded	High-byte runs do not decode as this bytecode. Early pattern counts (61, 17, 94, 62) were not handlers.
`tb80anm.sbk`, `tb80hyp.sbk`	0 decoded	Same. Strings and menus, not this handler format.

Of the 76 handlers in `9e954d01.tbk`, 75 are `buttonClick` and one is `EnterPage`. The 100 in `nida81.tbk` are all `buttonClick`. A handler whose name is not stored as plain text is not in those counts. The 11.5 export of `nida81.tbk` has eleven handler names. Ten of them are absent from this scan.

`openscript_analysis v1.0.0r9` reads this resource the same way: count at byte 0, id index, back-reference from the input, `0xFF` as a new character plus a count. It does not follow a back-reference that lands on another back-reference. The `EnterPage` handler does not need that chase. The release before it read the count from byte 2, skipped `0xC0`–`0xDF`, and repeated the previous output byte.

Lesson books store the handler name as plain text and compress the body. `tb80r.sbk` compresses the name as well. Encoding rules match. "No script at all in any system book" is too strong for `tb80r.sbk` and still true, as decoded text, for the four `nc81*` books.

14.5 What the courses actually do with it

The common subset is `go to`, `send`, assignment, `put into`, `set to`, `get`, increment and decrement, nested `if` / `else` / `end if`, `step from` / `to`, and `name of this book` / `this page`.

The clean-room engine also accepts `send NAME to TARGET`, a `get` that is a call, `forward to TARGET`, `break NAME`, and `PROP of TARGET`. Predicates such as `is not in` are recorded rather than evaluated. `local` and `system` declarations are ignored. Property writes are not applied to a live object. The expression evaluator copies `FUN_1008_2d31`: `+ - * /`, `abs`, `sqrt`, `log`, `exp`, `sin`, `cos`, `atan`, `asin`, `acos`, and otherwise leaves the original text. That function is not the record reader.

The page-by-page jumps are in chapter 13. Navigation is not "only a quoted page id." P20000 goes to P20001 because the handler says so.

14.6 Not done

A short script compiled in Instructor 8, saved, and expanded with ordinals 1 and 3 would lock the 994-name table against a known source. That test is written down and not run. Nested property expansion, opcodes past the common subset, and `forward` into a system-book handler are unimplemented or only sketched. The CMP expand host that died in `TB80W16` (`001c:00000015`, segment not present) is parked and is not the NIDA open path.

15. System books and the NIDA shell

`.sbk` and `.tbk` are the same container. Most of the Asymetrix `.sbk` files are resource containers: an object-type name table and menu resources. A few carry compressed handlers. The NIDA lesson still opens if the system books are removed. What breaks is the shell: blue screen, resize, and the menu. Renaming `nida81.tbk` removes the splash and reports that the shell is not a book. The lesson can still be entered.

15.1 Type names

The name table, in the order recovered from the system books, is: ComboBox, Stage, Picture, Hotword, RecordField, PaintObject, Curve, AngledLine, Pie, Arc, IrregularPolygon, Polygon, Line, RoundedRectangle, Ellipse, Group, Field, Button, Rectangle, then Page, Background, Book.

In `nc81base.sbk` that table sits around file `0x15E4`–`0x1ADC`. Curve's type id is `0x05C7` at file `0x1810`. The schema spelling `IrregularPloygon` is the 11.5 XSD, not this table.

These names are not yet tied to the kind byte inside a `14 00 00` chunk. `TB80UTL` holds the strings. It has not been shown to be the function that classifies a chunk.

15.2 NIDA shell

`nida81.tbk` is a real book. 43 objects. Pages `P20000` through `P22130`. It does not name lesson ids. It has 100 `buttonClick` handlers in the same bytecode as a lesson. P310 is Minimum Requirements (586, 133 MHz, 12 MB RAM, 1 GB, mouse, Windows 98). P20001 is the small menu. P22130 is Test Console Setup.

`9e954d01.tbk` at file `0xB1A96` names `tb80r.sbk`, `nc81trnr.sbk`, `nc81base.sbk`, `nc81recs.sbk`, and `nc81test.sbk`, and it calls `initializebase`, `initializerecs`, `initializetest`, `openSession`, and `gotoPageList`. It also jumps to `nida81.tbk` and to `93954302.tbk`.

Thought. Those initialize names have not been watched as a call log, and the order in `toolbook.lst` has not been watched live. `version.ini` in the set is `03.001.0006`, dated 04-18-05. `CAILT.DAT` maps course ids to Navy CDs. `9E954` maps to DIE. `STDAPPS.LST` is a short encoded list, not a book.

15.3 Asymetrix `.sbk` tiers

Sizes are the files that were triaged. JBO is at `0xC00` unless noted.

Tier	File	Size	What it is for
Core	`tb80r.sbk`	954 KB	Reader book. 66 objects. 33 compressed handlers (chapter 14). Help, bookmarking, logging, path manager, popup page manager. Not the engine.
Core	`tb80.sbk`	3.1 MB	Large core system book. Type table present. Not fully extracted.
Core	`tb80a.sbk`	2.4 MB	Author-side companion. Not on playback.
Core	`tb80i.sbk`	619 KB	Instructor system book.
Core	`INST80.SBK`	70 KB	Installer book. Referenced by Inst80. Names `tb80r`.
Useful	`tb80hyp.sbk`	141 KB	Hyperlink and help. The binary scan finds no `to handle` text. The TBKX export does.
Useful	`tb80anm.sbk`	64 KB	Path animation. The binary scan finds no `to handle` text. The TBKX export does.
Useful	`tb80aicc.sbk`	86 KB	AICC strings. Not shown to be required for a local page.
Useful	`tb80actr.sbk`	208 KB	ActiveX runtime.
Parked	`tb80acta.sbk`	2.1 MB	ActiveX author.
Parked	`tb80axf.sbk`	618 KB	ActiveX related. JBO at `0xE00`.

Tier	File	Size	What it is for
Not on the local path	`tb80web.sbk`	1.1 MB	Web.
Not on the local path	`tb80ftp.sbk`	243 KB	FTP. JBO at `0xE00`.
Not on the local path	`tb80x.sbk`	256 KB	Unclassified specialty.
Not on the local path	`tb80lmx.sbk`	62 KB	Small. JBO at `0xE00`.

An early triage said system `.sbk` files do not contain OpenScript. That is superseded for `tb80r.sbk`. The 21 October TBKX exports of `tb80r`, `tb80anm`, and `tb80hyp` also contain the handler source. A binary scan that looks for the words `to handle` still misses them. The other rows above still have no exported handler text.

The four NIDA books `nc81base`, `nc81recs`, `nc81test`, and `nc81trnr` are password-protected resource containers. Runtime opens them without the password. The author path is the one that asks. The password is not required to read the records. The ciphertext those books store is chapter 16.6.

15.4 Trainer and session DLLs

`nc81int.dll` and `nc81rec.dll` are 32-bit PE. They are not Asymetrix.

`nc81int.dll` exports 47 trainer calls. `IntCheckComm130E` opens a port at 9600 8O1, sends a probe, and returns `0x3D` on match or `0x3C` on failure. Families: 130E, TD2000, 5050, 205E.

`nc81rec.dll` writes `0x46`-byte records to `SESSION3.REC`. Exports include `RecStartSession`, `RecCloseSession`, `RecBookmarkSession`, `RecWriteResponse`, `RecReadVariable`. `initializeRecs` on the shell clears responses when a session loads. That last sentence is inferred from the shell script, not from a live trace of the DLL.

15.5 `linkDLL` versus a system book

A system book adds script above the book. A `linkDLL` adds commands, and only when a handler links it. The set that shows up that way is `TBDLG`, `TBWIN`, `TBDOS`, `TBREG`, `TBDB3`, `TBPDX`, and `TBCBT`. `TBWIN` is `INIT_TBKWIN` and `TBM_EVALUATE`. Neither `TBWIN` nor `TB80NET` walks page records. `TBCBT` is the CBT scoring extension and the password cipher. It is not the record reader.

15.6 Providers are not `sysBooks`

Segment 1278 loads class providers: a TBX DLL, a VBX DLL, a 32-bit TBX DLL, or another ToolBook book. It does not push `sysBooks`. The names sit on the book's class table. `a613` names `TB80ANM.SBK` and `TB80R.SBK`. When the provider is a book, or a TBX calls `REGISTERSYSBOOK`, the book goes on a bound list. The Author program adds that list to `sysBooks` through `CDBADDBOUNDSYSBOOKS`. The note is `research/class-providers.md`.

15.7 The 11.5 names are not these books

The ToolBook 11.5 common-file set uses the same filenames as the Instructor 8.1 system books. The files are not those books. Do not replace the rows in 15.3 with them.

An inventory of that set on 24 October 2026 (opentoolbook8) found two stubs:

Bytes	Names	Version resource
29,147	62 names, one hash. Includes `tb105hyp.sbk` and `tb80actr.sbk`, `tb80aicc.sbk`, `tb80anm.sbk`, `tb80ftp.sbk`, `tb80hyp.sbk`, `tb80r.sbk`.	TB110 `11.00.37`
29,145	`tb110actr.sbk`, `tb110aicc.sbk`, `tb110anm.sbk`, `tb110ftp.sbk`, `tb110hyp.sbk`, `tb110r.sbk`.	TB115 `11.50.12`

On both stubs the 16 bytes at `0xC00` are zero. The object-store header appears once, near offset 16440, not at `0xC00`. Neither stub returned book script. The 29,145-byte file shares a prefix with `tb115hyp.sbk` and is not that book cut short. The Instructor copies of `tb80anm`, `tb80hyp`, and `tb80r` stay 64,023, 140,613, and 953,705 bytes. Those three are not in the 11.5 archives.

16. The password cipher

The NIDA system books are password-protected. A lesson runtime opens them without presenting the password. Instructor and the author path are what ask. This chapter records where the cipher is and how the addition works. Section 16.6 is the empty-key branch and the ciphertext stored on the NIDA system books. The keyed 10-byte path in 16.2 is still open. The short strings tried against a stand-in key are not that password.

16.1 Where it lives

`TBCBT.DLL`, NE segment 7. Ghidra's name for that segment is `Code7`, base `1030`. The usual map is `0x1000 + (N - 1) * 8`, so segment 7 is `1030`. NE segment numbers stay fixed. Live selectors change. Segment 7 was not mapped in the SoftICE session that listed TBCBT code segments `01`, `0E`, `02`, `0B`, `08`, `0D`, `04`, and `03`. Those loaded segments are stubs and KERNEL/USER imports, not the cipher.

Exports on this path: `MAKEKEYEDPASSWORD`, `MAKEKEYEDPASSWORDEX`, `KEYEDPASSWORDENCRYPT`, `KEYEDPASSWORDENCRYPTEX`, `ENCRYPT`, `ENCRYPTEX`, `DECRYPT`, `DECRYPTEX`, `DECRYPTFILE`, `CREATEENCRYPTEDFILE`, `READENCRYPTEDFILE`, `WRITEENCRYPTEDFILE`.

The book path is `MAKEKEYEDPASSWORDEX` then `KEYEDPASSWORDENCRYPTEX`.

`TB80RCA.DLL` is Asymetrix UI (ASYMBITMAP, STATIC, TABS, spell, palette, BMR). It is not on the book-open path and not on the password path. `TB80ATH.EXE` is the 16-bit author and runs under NTVDM. The 32-bit pieces on a lesson launch are the launcher and the thunk (`TB80RUN`, `TB80W32`, `TB80MAIN`), not the cipher.

16.2 Three addition schemes

From `MAKEKEYEDPASSWORDEX`, `KEYEDPASSWORDENCRYPTEX`, and `FUN_1030_1046`. Inverse of each is `plain = (cipher - key) & 0xFF`.

Scheme	Add	Key byte
`(0xA01, 0x300)`	`toupper` of the key byte	From a 10-byte table. Index is `local_16 % 10`, then `+ 0x32`.
`(1, 0x400)`	`toupper` of the key byte, wrap at `0xFF`	Same table.
`(1, 0x700)`	The raw key byte, wrap at `0xFF`	Same table.

The live key is the 10 bytes that table selects. It is not the ASCII the user types. A test that encrypts `"nida"` or `"adin"` with a stand-in key `A`–`J` only checks the adder. It does not recover a book.

After a failed password dialog, search memory for the typed ASCII and for the constants `0xA01` and `0x300`. That finds the call site even while segment 7 is unloaded. Clicking OK and accepting "Wrong password" is the intended setup for that search. The dialog procedure itself is an ordinary Windows dialog.

16.3 What a with-password file difference is

A small book saved with a password and the same book saved without one differ from file offset `0xCEC` onward. The tails do not reconverge. The extra length is 1526 bytes of shift, not 1526 bytes of cipher text sitting at the end.

The local blob at the first difference begins:

```
c9 18 03 00 00 00 00 00 14 00 00 f0 00 00 77 2e
18 00 77 2e 6e 00 00 00 0c 45 00 00 7a 45 00 00
54 00 16 0d 00 00 0b 00 98 05 56 0e 00 00 00 00
```

The stand-in key does not decrypt it. Brute-force hits that came back short (`vgah`, `ap1`, `db9`, `iud`, `lxg`, `t57`, `vga`, `3p1`, `3ro`, `9e4`) are not the book password. They are discarded.

16.4 What is still open

This section is the keyed path only. The empty-key branch is 16.6 and does not use this table. The 10-byte key used on a real NIDA `.sbk` has not been captured. Segment 7 has not been stepped on a failed password. Until that capture exists, do not treat a decrypted string from the stand-in key as a finding.

16.5 Author password does not load TBCBT

On 16 October 2026 the author asked for `C:\TBK\nc81base.sbk`: "Please enter the password to gain Author level access." The next box said that password was not correct. The window title stayed `(Untitled)`, then became `sssssssssss.tbk`. The system book did not open.

`sssssssssss.tbk` became a password-protected book in that same session. `TBCBT.DLL` was not in the module list. The Extensions dialog on that book is the machine's OLE registry (`LMRT.DLL` and the other registered controls). It is not the `linkDLL` set, and `TBCBT` is not an entry on it.

Writing the author password stores the property block in section 16.3. It does not map segment 7. `TBCBT` is present only when a handler `linkDLL`s it. A new book has no such handler. A refused system book never runs its script, so it does not link the DLL either.

The stops while `sssssssssss.tbk` was current:

Stop	What it did
`??0TASKCOLLECTION`	UTL constructor. One `0x3C`-byte block. The flags word pushed is `0x2042` (movable, zeroed, shared). Called twice, two incoming selectors.
`GETTBKINIPROFILESTRING`	Reads `tbk.ini`. `AX` held a pointer (`AEA0`, `AD62`) when a string was found, and `0000` when it was not.
`GETTBKINIPROFILEINT`	Same file. The default at `[bp+6]` was `0x32D0`, then `0x3304`. `AX` came back `0000` both times.
`TB80ATH 4E67:0642`	The author EXE. `AX` was `4E2F`, so the `je` was not taken. Stores ES, CX+`0x100`, SI, DI, BX (`0x80`), and DX at `DS:305C`.
`GETCHARINFO`	Returns the cached dword at UTL `DS:0518`. That dword was zero.
`GETSTRMGRSTRING`	String id at `[bp+6]` was `0001`, then the function decrements it. Id `0` is the empty case.
`GETDATA@TASKCOLLECTION`	Lookup. Arguments were `0000` and `42F7`.
`OBJFILEOPEN`	BAS, `enter 025A` (602-byte frame). This is the open of the `.tbk`. `BX` was `0x50`, `CX` was `0x32D`.

After the status line "The task has completed running", every register was zero and the assembly pane was `????`. The `OLEUI*`, `DATASTUB*`, and `COPENSCRIPT` names on that screen are the export list of a dead task. They did not run.

16.6 Empty key

ENCRYPTEX with an empty key does not use the 10-byte table in 16.2. The nine bytes at DS:0x28 are BMWBMWBMW, indexed by position modulo 9. A letter encrypts as ((K + c - 0x82) mod 58) + 0x41. The inverse is what DECRYPTEX does: subtract K, and if the result is negative add 58, then add 0x41. cipher/empty_key.py is that pair. It is not the keyed branch.

Known. 62 6d 77 62 decrypts to aaaa. One a encrypts to 62.

Known. nc81base.sbk stores 62 70 49 6a 7a at file 0x1F6C. Those bytes are the author password. The same five bytes are in NC81BANK.SBK, nc81recs.sbk, nc81test.sbk, and nc81trnr.sbk. The password is not required to read the records. It is not printed here.

The keyed path in 16.4 is unchanged.

16.7 The lesson books

9e954d01.tbk and nida81.tbk store 0 in the first word of the book record. Neither has the password block this chapter describes. The author program that asks for the password is not in this archive. Chapter 16.5 already recorded that the prompt does not load TBCBT.

17. Modules

Eight modules were counted for pattern coverage on 2026-10-11: 10,991 unique functions, 1,918 nameable by pattern (17.5 percent). Confirmed renames after the later digs are 112: BAS 20, BMP 14, RTM 9, and the rest across CMP, UTL, W16, LNL, MM, RCR, FLT, NET, JPG, and VBX. Pattern coverage: RTM 32.7 percent, UTL 41.1, BMP 26, BAS 17.4, W16 17.3, LNL 12.2, CMP 5.6. A pattern name is a candidate. The scripts in rename/ are what was applied. Naming another thousand StrCpy wrappers will not find the record reader.

17.1 Launch

Piece	What it is	What it is not
TB80RUN	32-bit launcher. Spawns TB80NET.EXE with the book on the command line.	Not an in-process thunk, and not the parser.
TB80NET.EXE	16-bit NE. Network provider: zlib, TBKNET3WNDPROC, TBK.INI. Class TbkNet3TB80.	Does not parse pages.
TB80RTM.EXE	16-bit shell. TBKMain, TBKClient, TBKChildWindow, TBKReserved_1, SchedulerWndTB80. WM_PAINT calls CLIENTTOPAGE, then BAS ordinal 26.	Does not draw shapes. A lesson click arrives on TBKClient, not TBKChildWindow. BMSG does not see that queue.
TB80MAIN.EXE	23552-byte 32-bit stub. Calls TB80RMT CreateRemoteCallServer and pumps keys. Failure text: "TB80MAIN: Loading ActiveX support failed."	Not the book engine.
tbload.exe	Finds ZZZTBKBOOKLOC via MINSTB32.dll (FUN_00405460). Virtual call at 00406aa1 inside FUN_00406a70 (vtable+4).	The callee is unresolved. Thought: first handoff out of the 32-bit search.
TB80W32.DLL / TB80W16.DLL	Thunk halves, about 246 and 406 exports. W16 is also the DoCmd* bus (DoCmdDraw, DoCmdMouseDown).	W16 setup (platform, OLE, 32-bit LoadLibrary) is not book open.
TBFILE32.DLL	Long filenames for the 16-bit engine.	
TB80F32L.dll	Flat 32-bit entry and the common-dialog hook.	
TB80RMT	Remote-call server used by TB80MAIN.	Not on the path that opens a lesson.
PHYSSIZE	A 1.5-era index.	Not in these 8.1 books.

The engine calls 16-bit USER, GDI, and KERNEL. A log that says USER32 is missing is not a broken install. The images in hand are 16-bit NE. ToolBook 9.0 is the 32-bit product. A correspondent's claim that 8.1 is 32-bit and needs Windows 10 is rejected by the binaries (claim C-006).

17.2 Book server and neighbors

Module	Role	Not
TB80BAS.DLL	Objects, properties, painters, twip scale, file open. 606 exports. CDBINIT ordinal 1, CDBFILEOPEN 8, CDBVIEWPAGE 15, CDBVIEWBKGND 18, CDBPAINTCLIENT 26, CDBSENDCREATE 51 (author). The CDB* / OBJ* split is chapter 20.	Not the window.
TB80CMP.DLL	Compile and expand. Resources 0x77, 0x78, 0x70.	Not the page walker. 5.6 percent pattern-nameable because it is a compiler.
TB80UTL.DLL	Type-name strings, string manager, metrics, heaps.	Not the 14 00 00 reader.
TB80RCR.DLL	Controls, mouse hook, effect objects, button drawing. Init is FUN_1010_0000.	Not classified as the book reader. The init does not open a file.
TB80ENU1 / ENU2	English strings, 16-bit and 32-bit.	Not code that opens a book.
TB80BMP.DLL	DIB, metafile, FX blit. Ordinal 3 is the bitmap reader. Lz77Expand is FUN_1000_0031.	Not a picture store. The widget still does not name the bitmap.
TB80JPG.DLL	JPEG. Version resource still says TB60JPG.DLL.	
TB80LNL.DLL	MCI, wave, MMIO. The notify callback is Lionel.	
TB80MM.DLL	tbkmmNotify, imageNotify, mciNotify, timerNotify.	Does not classify objects.
TB80OLE / OCX / ACX / VBX	Clipboard, controls, automation.	Not on the Logic Families paint path.
TB80FLT	Filter registration. FUN_1028_2680 calls TBREGISTERFILTER. The 0x14 there is a type-code limit.	Not the reader.
TB80RCA.DLL	Author UI widgets.	Not book open, not the cipher.
TBCBT.DLL	CBT scoring. Segment 7 is the cipher (chapter 16).	Not the reader. Not loaded when the author password-protected ssssssssssss.tbk (section 16.5). Not loaded as segment 7 in the session that listed the other code segments.
TBDLG, TBWIN, TBDOS, TBREG, TBDB3, TBPDX	linkDLL commands.	Not page walkers.
DIBUTIL.DLL	DIB helper beside BMP.	Not the object format.

Author binaries TB80EDT.DLL and TB80.EXE are not in this set. The runtime half of the author contract is visible: BAS exports CDBEDITBOOK, CDBVIEWBOOK, CDBISVIEWERINGENERICEDITMODE, CDBSENDEDITLEAVETOALL*, and TBXAUTHORMODE. RTM calls GETAUTHORLOCALIZEDDLL. TB80ATH.EXE is the 16-bit author, seen under NTVDM. That is bookmarked, not solved.

17.3 Closed as the reader

Candidate	What it actually is
CDBSENDCREATE (ordinal 51, 12c8:00BC)	Author create and rewrite.
FUN_12c8_0ef8	Copies four words on that author path. The source is not the disk record.
FUN_1028_2680	Filter registrar.
FUN_1008_2d31 (switch 1008:2DC7)	OpenScript math.
TB80RTM	Paint handoff only.
CDBCHUNKFIND / LAST / END / CDBSKIPSEP	Text separators.
VALUENEWLONG and the 4267 constructors	Value constructors.
FUN_10b0_00a6	Type dispatcher. Code 0x14 is a type code.
12e8:0000	OLE object clone.
BAS segment 182	39 far calls with a zero segment and no reloc. Not book open.
BAS segments 1018, 1278, 1478, 14b8, 1508, 10d0, 1030, 14a8, 1160, 1070	Sampled. No stream walker. 1160 is debugging (CDBGETNEURONURL, CDBATTACHBREAKADDRESS). 1018 creates pens, brushes, and bitmaps.
A static search for the bytes 14 00 00	Thirteen undisassembled runs contain the word 14 00 as immediates, MOV CX, 0x14, or jump displacements. Not a far-pointer compare that builds an object.

17.4 Ghidra

Two NE loader bugs were fixed for this work: the entry-table segment byte was treated as signed, and `ne_cbenttab` was ignored. `TB80BAS` is the image with entry segment bytes at or above 128. `toolbookunpacker` clears the Optloader self-load flag and does not patch the entry table. The Ghidra patch does. An honest BAS image loads in the patched Ghidra 12.1.4 build.

Force-disassemble: walk undefined `C8` (`ENTER`) bytes first, then the prologue pass. The prologue pass accepts a real `RET` / `RETF` before treating the following byte as code, and it only creates functions at entry-like symbols. They do not name anything.

`SENDMESSAGE` (321 hits in one RTM listing) and `LSTRLEN` (282) have mangled arguments. That is a decompiler problem. It wants a fresh decompile, not another reading of the same HTML.

`maps/bas-ne-to-ghidra.md` and `maps/rtm-ne-to-ghidra.md` are the segment crosswalks. `maps/wdw-to-ghidra.md` is one boot's selectors. None of the three is a narrative. Appendix K of the reference manual is the procedure those maps belong to.

17.5 RCR, from the 16 October stop

There is no separate HTML listing. The decompile is `Toolbook/_decompiles/TB80RCR.DLL.c`. It has 293 functions: 191 still named `FUN_`, 53 `ASYMTBKFX*`, 13 `ASYMCTL*`, 10 `BMR*`, 8 `ASYMMTRK*`, plus help, `WEP`, and `BTNPROCBITMAPBUTTONPROC`. The Watcom list is that export set. The names are definitions in this file, not unnamed functions waiting to be matched.

The stop was `TB80RCR.DLL`, selector `3C4F`. `ASYMTBKFXINIT` at `493A` is the decompile at line 25267: `FUN_1008_aa62`, then return 1. The far call `43EF:AA62` is that first call. `ASYMMTRKINIT` at `2E0A` is line 23315. The `cmp` of word `0044` is `DAT_1020_0044`. Zero means first time. The following `push 2042h` is the `GLOBALALLOC` flags. The listing allocates `0x84` bytes, locks them, registers two window messages, and calls `SETWINDOWSHOOKEX`.

Those two are not the first instructions of the DLL. `entry` (line 9587) does `LOCALINIT`, `GETVERSION`, and `int 21h`, then `FUN_1008_d7ca`, which calls `FUN_1010_0000`. That function's order is `LOCALINIT`, `HELPINIT`, `ASYMTBKFXINIT`, `ASYMMTRKINIT`, and if the mouse init returns 0, `FUN_1010_c79e`. `WEP` calls `ASYMTBKFXEXIT` and `FUN_1010_ca82`.

`BMRDRAWBUTTON`, `BMRDRAWBEVEL`, `BMRDRAWSHADOW1`, `BMRDRAWSHADOW2`, and `BMRDRAWSTATUSBAR` are the drawing. They are button faces and bevels. The init that was stopped does not open a book.

A later stop in the same session was `43EF:2992`, `ASYMCTLPAUSE` (line 3606), not `ASYMCTLPAUSED` (line 3656). The prologue has not run, so `DS` is still `3327`. The watch `3327:1B02 = 5341` is not the word the compare reads. After `mov ds, ax` the compare is `41FF:1B02`, which is `DAT_1020_1b02`. `ASYMCTLPAUSE` calls `FUN_1008_34de` and, on a hit, `UNHOOKWINDOWSHOOKEX`. `ASYMCTLPAUSED` only returns 0 or 1. `ASYMCTLISAPPREGISTERED` at `2ACE` is the same lookup: `push 0`, `push [bp+6]`, `call near 34DE`. `ASYMCTLGETPROPERTY` at `2D40` calls `GETCURRENTTASK` and then that lookup.

18. Where every loose sheet went

The absorbed sheets now live under `tbworkingpapers/history/sheets/`. A row that says `tbwp0001/…` means `history/sheets/tbwp0001/…`. The manual source stays at `tbworkingpapers/sheets/tbwp0030/`. `tbwp0019`'s notes, and the seventeen renamed JSON files that still read, are in `history/sheets/tbwp0019/`. The rest of that dump does not read. It is still `tbworkingpapers/tbwp0019/`. `who-is-who` is not in this index and is not in this repository. The shelf is `tbworkingpapers/history/miscellaneous/`. Binary password deltas stay in the working

papers, under `history/sheets/tbwp0027/`.

A row marked superseded means the sheet still contains a sentence this record closed. Read the sheet for the function body if you need it. Read this record for the claim.

Loose sheet	Fate	Where the claim lives
`tbwp0001/00-architecture.md`	kept	Chapters 3 and 17.
`tbwp0001/01-dll-catalog.md`	kept	Chapter 17.
`tbwp0001/02-open-questions.md`	superseded	Chapter 10. The reader is still unnamed; the create-path wording in that note is superseded.
`tbwp0001/ToolBook_Architecture.md`	superseded	Chapters 3–5. The book-open flowchart is superseded by chapter 4.
`tbwp0001/ToolBook_Chapter.md`	superseded	Chapters 3–5 for layout. Book-open flowchart that calls CDBSENDCREATE is superseded by chapter 4.
`tbwp0001/naming_summary.md`	kept	Chapter 8 and chapter 17. Counts 112 confirmed.
`tbwp0001/openscript_notes.md`	superseded	Chapter 14. The early italic/systime conclusion is superseded by the back-reference fix in the same file.
`tbwp0001/tbworkingpapers-pointer.md`	index	Chapter 0. Pointer only.
`tbwp0002/README.md`	kept	Chapter 7. Maps are NE-segment:offset and are not loaded as modules.
`tbwp0002/bas-create-path.md`	superseded	Chapter 4. CDBSENDCREATE is not the runtime reader.
`tbwp0003/p1040-breakdown.md`	kept	Chapter 13.1. TBX expansion of P1040.
`tbwp0003/p1040-tbk-raw.md`	kept	Chapter 13.1. Bounds, fills, blob offsets.
`tbwp0003/tb80r_compression.md`	kept	Chapters 14 and 15. 33 handlers, prefix also compressed.
`tbwp0003/tbx-vs-tbk-analysis.md`	kept	Chapters 2 and 13. Bounds in 8.1, Vertices in the export.
`tbwp0004/README.md`	kept	Chapters 2 and 5. Specimen graphics. Renders are not re-shipped here.
`tbwp0005/README.md`	kept	Chapters 6 and 9. Engine subset.
`tbwp0006/foollege-hierarchy.md`	kept	Chapter 9. Foollege is not a running server.
`tbwp0006/inventory.md`	kept	Chapter 13.7. Tag counts. Folder names are not runtime page names.
`tbwp0007/bounds-extractor.md`	superseded	Chapters 5 and 13. The 14 00 00 01 becomes 11 hypothesis in the header era is not true (WV-010).
`tbwp0007/content-rectangles.md`	kept	Chapters 5 and 13.1. Bounds-only, marker 01 00 00 04.
`tbwp0007/core-sbk-confirmation.md`	kept	Chapter 15.
`tbwp0007/denny-exchange-summary.md`	kept	Chapter 2 for the Save To XML steps. The 32-bit claim in that exchange is rejected (C-006). Personal names are not copied.
`tbwp0007/ghidra-research-list.md`	superseded	Chapters 10 and 17. Create-path items in that list are superseded.
`tbwp0007/minor-decompile-notes.md`	kept	Chapters 2, 5, and 17. Graphic property ids `0x4097`–`0x409B`, `LOCKPATTERN`, `layer` at `0x4059`. The page-blob slot for the bitmap id is still open.
`tbwp0007/new-dumps-analysis.md`	kept	Chapters 7 and 12. A dump is real only when it contains known book strings.
`tbwp0007/page880-comparison.md`	kept	Chapter 13. Page880 is P1040, IdNumber 880.
`tbwp0007/processing-log.md`	method	Chapter 18. A log, not a separate finding.

Loose sheet	Fate	Where the claim lives
tbwp0007/sbk-triage.md	superseded	Chapter 15. The 'no OpenScript in any sbk' line is superseded for tb80r.sbk only.
tbwp0007/schema-usefulness.md	kept	Chapter 2. Bounds required, Vertices optional, IrregularPloygon typo.
tbwp0007/tbk-tbx-comparison.md	kept	Chapters 2 and 13.
tbwp0007/tbx-schema-notes.md	kept	Chapter 2.
tbwp0007/template-triage.md	kept	Chapter 15. Templates are not the NIDA playback path.
tbwp0007/waiting.md	kept	Chapter 10. Waiting list folded into the ranked gaps.
tbwp0008/cdbsendcreate-sites.md	superseded	Chapter 4. Callers are author-side.
tbwp0009/bas-html-extracts.md	superseded	Chapter 4. Extracts that treat ordinal 51 as runtime are superseded.
tbwp0009/html-coverage.md	kept	Chapter 17. HTML ?? is not absence.
tbwp0010/group-vs-offset-table.md	kept	Chapter 13.4. The word table is an index, not a Group.
tbwp0011/bas-xref-walk.md	superseded	Chapter 4. Cross-references that start at ordinal 51 are the author path.
tbwp0011/cdbsendcreate-and-nida81.md	superseded	Chapters 4 and 15. Author-only correction wins.
tbwp0012/new-html-listings.md	kept	Chapter 17. TBCBT, TBDB3, TBDLG, tbload, TBWIN surveyed.
tbwp0012/scaletwips-and-dispatcher.md	kept	Chapters 4 and 5. FUN_1008_01dd and the type dispatcher.
tbwp0013/14-00-00-pattern-search.md	superseded	Chapter 4. No literal compare found. Author-path sentences superseded.
tbwp0013/new-html-survey.md	superseded	Chapter 17. Survey. Author-path readings superseded.
tbwp0013/paintpolygon-pointer-search.md	kept	Chapter 5. No static far pointer to 1408:0376.
tbwp0013/switch-1008-2dc7.md	kept	Chapters 6 and 14. Math, not the reader.
tbwp0013/tbload-zzztbk-walk.md	kept	Chapters 2 and 17. Marker search and the unresolved vtable+4.
tbwp0014/14-00-00-in-undisassembled.md	kept	Chapter 4. The hits are immediates and displacements.
tbwp0014/approach-correction-undisassembled.md	kept	Chapter 4. Do not close a function for lacking the literal.
tbwp0014/author-side-traces.md	kept	Chapter 4. Author exports are real and not book open.
tbwp0014/bas-seg182-leftover.md	kept	Chapter 4. Not book open.
tbwp0014/code94-ole-clone.md	kept	Chapter 4. 12e8:0000 is an OLE clone.
tbwp0014/fun-1028-2680-is-filter-registrar.md	kept	Chapter 4. Closed.
tbwp0014/high-priority-status.md	superseded	Chapter 10. The disk-to-11 expansion row is not true.
tbwp0014/instructor8-compile-test.md	kept	Chapter 14.6. Written, not run.
tbwp0014/low-priority-status.md	kept	Chapter 10.
tbwp0014/medium-priority-status.md	kept	Chapter 10.
tbwp0014/new-html-survey.md	superseded	Chapter 17.
tbwp0014/openscript-token-map.md	kept	Chapter 14. What compile-plus-expand would lock.
tbwp0014/paintpolygon-table-static-limit.md	kept	Chapter 5. Table is runtime-built.
tbwp0014/supporting-modules-content.md	kept	Chapter 17.
tbwp0014/supporting-modules-useful-for.md	kept	Chapter 17.
tbwp0014/undisassembled-regions.md	kept	Chapter 17. Forced disassembly does not name them.
tbwp0014/undisassembled-triage.md	superseded	Chapter 4. Author-path readings superseded.
tbwp0015/README.md	method	Chapter 17. C8 pass, then prologue pass.

Loose sheet	Fate	Where the claim lives
tbwp0015/win16-merged/README.md	method	Chapter 17. Duplicate of the win16 note.
tbwp0015/win16-merged/scripts/ForceDisassembleC8.md	method	Chapter 17. Duplicate.
tbwp0015/win16-merged/scripts/ForceDisassemblePrologues.md	method	Chapter 17. Duplicate.
tbwp0015/win16/README.md	method	Chapter 17. Win16 support notes for the unpacker.
tbwp0015/win16/scripts/ForceDisassembleC8.md	method	Chapter 17. Method. Run before the prologue pass.
tbwp0015/win16/scripts/ForceDisassemblePrologues.md	method	Chapter 17. Method.
tbwp0016/openscript-expression-path.md	kept	Chapters 6 and 14. FUN_1008_2d31.
tbwp0016/page-background-index.md	kept	Chapters 4.5 and 10. Background id is not a word in the page record that has been decoded.
tbwp0016/property-block-expansion.md	superseded	Chapter 4. FUN_12c8_0ef8 is author-side. The 01-to-11 mutation is not true.
tbwp0017/bas-segment-pass2.md	kept	Chapter 4. Segments sampled, no stream walker.
tbwp0017/bas-segment-triage.md	kept	Chapter 4.
tbwp0017/side-quests-expanded.md	kept	Chapter 10.5.
tbwp0017/side-quests.md	kept	Chapter 10.5. Real, and not the reader.
tbwp0018/bas-decompile-4.md	superseded	Chapter 4. Same correction.
tbwp0018/bas-html-4-analysis.md	superseded	Chapter 4. Header correction stands: ordinal 51 is author-only.
tbwp0018/bas-html-4.md	superseded	Chapter 4. Same correction.
tbwp0018/current-goals.md	kept	Chapter 10.6. Order of work.
tbwp0018/decompile-polish-todo.md	method	Chapter 17. Polish is not the reader.
tbwp0018/dibutil-analysis.md	kept	Chapter 17. DIB helper, not the object format.
tbwp0018/foolbook-research-notes.md	kept	Chapter 9.
tbwp0018/record-reader-candidates.md	kept	Chapter 4.4. Candidates closed.
tbwp0018/rtm-3-triage.md	kept	Chapter 17.
tbwp0018/rtm-decompile-2.md	kept	Chapter 17. SENDMESSAGE / LSTRLEN arguments want a fresh decompile.
tbwp0018/rtm-deep-analysis.md	kept	Chapters 5 and 17.
tbwp0018/rtm-html-4-analysis.md	kept	Chapter 17. RTM is the paint handoff.
tbwp0018/rtm-html-4.md	method	Chapter 17. Listing note.
tbwp0018/rtm-html-undisassembled.md	kept	Chapter 17. ?? runs.
tbwp0018/rtm-nameable.md	kept	Chapter 8.
tbwp0018/rtm-paint-path.md	kept	Chapters 3 and 5. CLIENTTOPAGE then ordinal 26.
tbwp0018/rtm-triage.md	kept	Chapter 17.
tbwp0018/today-summary.md	method	Chapter 10. Day log of 2026-10-14. Later corrections win.
tbwp0018/unanalyzed-functions.md	kept	Chapter 10. Still unnamed is not the same as still a candidate.
tbwp0018/undisassembled-triage-2.md	superseded	Chapter 4. Author-path readings superseded.
tbwp0019/keepers.md	kept	Chapter 11. Enter sizes. Selectors are that boot.
tbwp0019/wdw-running-map.md	kept	Chapter 12. The spine.
tbwp0019/wdw-to-ghidra.md	kept	maps/wdw-to-ghidra.md. Lookup, not typeset.
tbwp0023/rtm-ne-to-ghidra.md	kept	maps/rtm-ne-to-ghidra.md. Already in the repo. Lookup.
tbwp0024/keeper-list-decompile.md	kept	Chapter 11. Line numbers in TB80BAS.DLL.c.
tbwp0024/keeper-list.md	kept	Chapter 11. Live addresses. CDBSENDCREATE removed.

Loose sheet	Fate	Where the claim lives
tbwp0024/ne-to-ghidra.md	kept	maps/bas-ne-to-ghidra.md. Lookup, not typeset.
tbwp0025/cdbsendcreate-is-author.md	kept	Chapter 4. The correction.
tbwp0026/bas-trace.md	kept	Chapter 12. 173 BAS shots.
tbwp0026/trace-review.md	kept	Chapter 12. Three traces, no reader.
tbwp0026/utl-trace.md	kept	Chapter 12. 53 UTL shots. No record walk.
tbwp0026/w16-trace.md	kept	Chapter 12. 11 W16 shots. No book open.
tbwp0026/wdw-break-syntax.md	kept	Chapters 7 and 12. @ not !, resume on is not a creation flag.
tbwp0026/wdw-trace.md	kept	Chapter 12.
tbwp0027/withpass-diff.bin	kept	Chapter 16. Binary stays in the working papers. Not committed.
tbwp0028/00-scaffold.md	index	Chapter 0. The scaffold this volume implements.
tbwp0028/chapters/00-how-to-read.md	kept	Chapter 0. Draft only. It is not the statement, and it lags the record after the author-dispatch tables and the graphic property ids.
tbwp0028/chapters/01-corpus.md	kept	Chapter 1.
tbwp0028/chapters/02-formats.md	kept	Chapter 2.
tbwp0028/chapters/03-runtime.md	kept	Chapter 3.
tbwp0028/chapters/04-objects.md	kept	Chapter 4.
tbwp0028/chapters/05-graphics.md	kept	Chapter 5.
tbwp0028/chapters/06-openscript.md	kept	Chapter 6.
tbwp0028/chapters/07-debugging.md	kept	Chapter 7.
tbwp0028/chapters/08-naming.md	kept	Chapter 8.
tbwp0028/chapters/09-modern.md	kept	Chapter 9.
tbwp0028/chapters/10-open.md	kept	Chapter 10. The chapters in this folder are the frozen draft. Where they disagree with record/, the record wins.

tbwp0029/00-official-record.md and the README.md at the root of the working papers are pointers. They are not findings. There is no tbwp0020, tbwp0021, or tbwp0022.

Dated folders already in this repository

Folder	Fate
trail/20261012/	Earlier consolidation, also typeset as the 9-page 15 October contents. Create-path wording that calls the reader a caller of CDBSENDCREATE is superseded by chapter 4. The dispatch tables, the registrar's property ids, and the painter offsets are kept there and in chapter 5.
trail/20261014/	Condensed working papers. 02-still-open.md still calls the reader the caller of CDBSENDCREATE. That sentence is superseded. 01-findings.md, 03-tbx.md, and 04-shelf.md now point at the record under the title. The rest of the open list matches chapter 10.
maps/ and rename/	Lookups and name lists. A trail line under the title says they are not findings.
trail/20261015/	00-how-toolbook-operates.md section 2 and 01-theories-of-operation.md section 1 still teach the author chain as runtime. Superseded by chapter 4. The nested 20261012/ inside it is a duplicate of trail/20261012/.
trail/20261016/00-consolidated.md	The short correction. This volume is the expansion.
rename/	The scripts chapter 8 refers to.
maps/	Segment crosswalks. Not typeset in the PDF.
tbpipeline/tbpipeline_bf/nehost/	The Unicorn host. Not in this repository. Segment dumps stay there, not here.

19. Heap, stack, and the resource manager

Date: 2026-10-16

Status: Side reading of `TB80BAS.DLL.c`, `TB80UTL.DLL.c`, and the 2026-10-15 WDW breakpass in `maps/wdw-to-ghidra.md`. Line numbers are the decompile. Live selectors are one boot. None of these functions is the unnamed record reader.

Claims

Id	Grade	Statement
H-001	Known	Three allocators. UTL `GHEAP*` is one Windows global block turned into a local heap. BAS `HEAP*` / `TBHEAP*` is the NearScript frame heap. `GLOBALALLOC` of `count * 0x76` in `FUN_1278_3be6` is the live-object array. They do not call each other.
H-002	Not true	"`HEAP*` / `TBHEAP*` hand out the `0x76`-byte object" and "there is no standalone `TBHEAPCLEARCRITSECT` definition." Both sentences were chapter 11. `TBHEAPCLEARCRITSECT` is decompile line 93984. No `TBHEAPALLOC` site passes `0x76`.
H-003	Known	The only static caller of `HEAPREGISTER` is `RESMGRCONNECT`. It passes the second of two `MAKEPROCINSTANCE` thunks. That thunk is stored at heap `+0x20` / `+0x22` and is an indirect call.
H-004	Known	The OpenScript value stack is tag `0x10` on that heap. It is not the CPU stack. A non-scalar push goes through `VALUETOSTRING` and the `CDBCHUNK*` comma split. That is why the stack and the chunk walkers shared a WDW batch.
H-005	Known	`RESMGRSENDMSG` is the resource dispatcher. `RESMGRENUM` calls a `code *` argument. Five WDW return addresses that call `VALUENEWRESOURCE` and `RESMGRGETREFCNT` are still unnamed.
H-006	Thought	The `0x303E`-byte heap object is the header plus a frame table of depth `0x400`. The arithmetic fits. The listing does not comment it.

19.1 Three allocators

The `0x76`-byte live object is chapter 4. `CDBNEWOBJECT` indexes it as `param_6 * 0x76`. The block that holds the array is `FUN_1278_3be6` (line 121049). Empty, it `GLOBALALLOC`s `param_2 * 0x76`. Otherwise it `GLOBALREALLOC`s to the same product. The free walk at line 120992 steps `0x76`. That path does not call `TBHEAPALLOC`.

`GHEAP*` is UTL segment 9, lines 9261–9390.

Export	What the body does
`GHEAPCREATE`	`GLOBALALLOC` of `param_2 + param_3 + 0x10`, then `GLOBALHANDLE`, then `LOCALINIT`. A non-zero `param_3` zeroes that many bytes from the symbol the listing calls `DAT_10f8_0010`. Whether that symbol is the new block was not checked. Failure calls `GHEAPDELETE`.
`GHEAPALLOC`	`LOCALALLOC`.
`GHEAPFREE`	`LOCALFREE`.
`GHEAPLOCK` / `UNLOCK` / `SIZE` / `REALLOC`	The matching `LOCAL*` call.
`GHEAPDELETE`	`GLOBALFREE` of the `GLOBALHANDLE`.

The BAS caller is `FUN_11a8_0000` (line 82614), after `GETFREESPACE` and `GLOBALCOMPACT`. It creates two heaps, `GHEAPCREATE` with sizes `0x2002` and `0x800`, stored at `DAT_1630_03b0` and `DAT_1630_03aa`. The `0x16E0` that the decompile prints as the first argument is the same extra the listing puts on `GLOBALALLOC` and `LOCALALLOC` throughout this file. It is not a third size. `FUN_11a8_03f8` (line 82888) and `FUN_11a8_087c` (line 83240) call `GHEAPALLOC` with size `0x40`.

The UTL export trace of 53 hits included `GHEAPCREATE`, `GHEAPALLOC`, and `LOADRESSTR`. `LOADRESSTR` (UTL line 7943, segment 7) loads a Windows string resource. It is not `RESMGR*`. That trace was not a record walk. Chapter 12 already says so.

HEAPREGISTER itself uses GLOBALALLOC for the directory and for each heap object. That is the third use of the Windows allocator, and it is not GHEAP*.

19.2 The script heap

BAS segment comment "Segment 63". The functions are FUN_11f0_*, lines 93127–94370. The decompile passes the string NearScript into this family. Treat that as the module's name for the heap, not as a type check, until FUN_11f0_2440 is read.

HEAPREGISTER (93127). If DAT_1630_11ae is null, GLOBALALLOC a directory of 0x2E bytes. Word 0 is capacity, set to 10. Word 1 is the count. Word 2 is 0xFFFF, the free-list sentinel. Each heap is GLOBALALLOC(0x303E). Word 0 of the heap is magic 0xCCDD. The first arena is FUN_11f0_280c(0x1000). The caller's far pointer is stored at heap +0x20 and +0x22. When the directory is full, GLOBALREALLOC grows the capacity by 9. The only static call is RESMGRCONNECT line 20089.

HEAPDEREGISTER (93236). If heap +0x20 / +0x22 is set, FREEPROCINSTANCE on it. Frees the arenas, unlinks the heap from the directory, and GLOBALFREEs the heap. Callers: the connect failure path (20127) and RESMGRDISCONNECT (20155).

HEAPCREATEFRAME (93315). Fails when the depth word at heap +6 is 0x400. Otherwise it carves at least 0x24 bytes from the current arena, or asks FUN_11f0_280c for another 0x1000. The frame slot is heap + depth * 10:

Offset	Contents
+0x82E / +0x830	Handler far pointer. HEAPGETHANDLER (94051) returns it.
+0x832 / +0x834	Arena far pointer.
+0x836	Caller BP with the low bit cleared. HEAPGETBPFRAME (94063) returns it.

0x82E + 0x400 * 10 is 0x302E. The error word TBHEAPALLOC writes is at +0x303C. Both sit inside the 0x303E object. That is H-006.

TBHEAPALLOC(size, tag, frameIndex, heap) (93620). Adds 4 for a header. FUN_11f0_25f8 rounds the sum. Rejects a zero round or a round >= 0xFFC5, and rejects a size that overflows when 4 is added. Those failures store 0x1FFF at heap +0x303C and return null. The block's first word is the tag. The second word is the frame index. The pointer returned is past those four bytes. Allocation walks the frame free list (FUN_11f0_21c4), grows it (FUN_11f0_1b18), and may start a new block (FUN_11f0_2790). Out of memory stores 0x1F7A. If the crit depth at heap +0xE is not zero, FUN_11f0_15aa must accept the block; otherwise the block is TBHEAPFREEd and the error is also 0x1F7A.

TBHEAPFREE (93800). The block is the user pointer minus 4. The frame index is the word at user-minus-2. It unlinks a 4-byte entry from the table at heap +0x18 / +0x1A. The scan gives up past 0x3FFF entries. The free of the bytes is FUN_11f0_22da.

The crit "section" is a nesting counter, not a Win32 critical section.

Export	Line	Effect
HEAPENTERCRITSECT	93893	If depth is not already -1, increment heap +0xE. Return the 32-bit cursor at +0x10 / +0x12.
HEAPLEAVECRITSECT	93909	May write the cursor back. Decrement depth if it is not zero.
TBHEAPSAVECRITSECT	93933	Copy depth and the cursor, three words.
TBHEAPRESTORECRITSECT	93953	Write them back. A non-zero saved depth does not move the cursor backwards.
TBHEAPCLEARCRITSECT	93984	Zero all three words.

HEAPFINDCDBVALUE (94907) calls TBHEAPALLOC(0x5A, 0xCDBF, ...). The size matches the node it then fills: the last word it writes is at offset 0x58. Every other tag in the table below is a small integer. Do not treat 0xCDBF as a type

code until those bytes are re-read. The same function's value-packing expression contains `*(uint *)0x0`, which is the decompiler losing a register.

Tags at the `TBHEAPALLOC` sites. The owner is the function whose body contains the call.

Tag	Size argument	Owner	Line
`0`	caller size	`VALUENEW`	85175
`0`	`local_8 + 0xC`	`FUN_11d8_1a28`	88923
`0`	`(n + 3) * 4`	`FUN_1368_0de0`, the stack chunk	154913
`9`	`param_1 + 3`	`VALUENEWSTRING`	87415
`0xA`	`8`	`VALUENEWDOUBLE`	86406
`0xB`	`4`	`VALUENEWLONG`	86282
`0xC`	`4`	`VALUENEWPOINT`	153277
`0xD`	`8`	`VALUENEWRECT`	153446
`0xE`	`7`	`VALUENEWDATE`	151703
`0xF`	`3`	`VALUENEWTIME`	151839
`0x10`	`0xC`	`VALUENEWSTACK`	154118
`0x11`	`param_4 * 4 + 0x16`	`FUN_11d8_0030`	87720
`0x12`	`0x100`	`VALUENEWCOLORTABLE`	155612
`0x13`	`param_1 + 2`	`VALUENEWICON`	153717
`0x19`	`10`	`FUN_11e0_1010`, `FUN_11e0_10e0`, `FUN_11e0_11dc`	90735, 90819, 90882
`0x22`	caller size	`FUN_1390_054a`, then `HEAPADDTOCLEANUPLIST`	157234

`VALUENEWCSTR` (87576) is the export in front of `FUN_11d8_0030`. The tag `0x11` call is in the helper, not in the export's own body.

19.3 Values, marks, and the indirect calls

A script value is not a pointer. `HEAPVALUETOPOINTER` (94011) takes the low 10 bits of the value (`value & 0x3FF`), scales by 2, and reads a selector word at `heap + index * 2 + 0x2E`. The offset half is the other argument. WDW's first instructions on that export are `and si,03FF` / `add si,si`. `HEAPVALUETOFRAME` (94020) is that lookup, then the word at the block's `-2`, which is the frame index. `FUN_11f0_0f76` checks the index is `< 0x400` and that `(block[-4] & 0x7FFF)` equals `value >> 10`.

`HEAPPOINTERTOVALUE` (94000) shifts the header word at `-4` left by 10 and mixes in a 10-bit field. The mix is the same broken `*(uint *)0x0` expression as in `HEAPFINDCDBVALUE`. The shift is in the listing. The low 10 bits of this direction are not a clean reading.

Mark and sweep:

Function	Line	What it does
`HEAPMARKFRAME`	94169	Sets bit 7 of the byte at pointer `-3`.
`HEAPMARKPOINTER`	94193	Walks the frame's blocks. Calls `FUN_11f0_2440`.
`HEAPSWEEPFRAME`	94304	Marks the frame's pointer list, then walks the mark table through `FUN_11f0_135a`.
`HEAPADDTOCLEANUPLIST`	93510	Returns 0 unless the header word `& 0x7FFF` is `0x22`.
`FUN_11f0_12ac`	94227	If the mark bit is set, clears it and keeps the block. If the type is `0x22`, `HEAPREMOVEFROMCLEANUPLIST`, then an indirect call.
`FUN_1390_0596`	157249	The matching free: remove from the cleanup list, then `TBHEAPFREE`.

The type-`0x22` call is not "the word at the block plus `0x2C`". `FUN_11f0_12ac` reads a far pointer from the first four bytes of the user data and calls the code pointer at that object plus `0x2C`. `FUN_1390_054a` stores the heap far pointer at user `+4` / `+6` and does not show what sits at user `+0`. The callee is unnamed.

`FUN_11f0_103c` (94071) locks the directory at `DAT_1630_11ae` and walks every registered heap. For a frame whose handler segment matches the argument, if heap `+0x20` is set, it calls that proc-instance, then `FUN_11f0_1160` may

rewrite handler slots. Static callers: FUN_10d0_07ee (line 51677, argument 1) and FUN_10d0_08ba (line 51737, argument 0). The proc-instance body is the thunk RESMGRCONNECT registered. This pass did not name the code inside the thunk. MAKEPROCINSTANCE in that function is decompiled against the CTbkSimpleOLEDispatch string pool, which is the same artifact chapter 4 warns about. It is not evidence the thunk is an OLE method.

FUN_1368_04ee (154384) is the stack marker. It checks the value with FUN_11f0_0f76, VALUEMARKs each element, and HEAPMARKFRAMEs each chunk and the header.

19.4 The value stack

Segment comment "Segment 110", FUN_1368_*, from line 154087. The context object is the script frame: heap far pointer at +4 / +6, frame index at +0xC.

VALUENEWSTACK allocates the header with TBHEAPALLOC(0xC, 0x10, frame, heap). Failure is CDBSETPLERR(..., 0x1F7A, 2, ...). Chunks come from FUN_1368_0de0. A request of 0 or more than 0x19 elements becomes 0x19. The chunk allocation is tag 0, size (count + 3) * 4. Each empty slot is written as type 1, subtype 0x400. The header holds the count, the first chunk, and the current chunk. HEAPPOINTERTOVALUE writes the value back to the caller. A second chunk is linked when the fill reaches 0x19.

Export	Line	Behavior
STACKGETSIZE	154518	Type 1 and subtype 0x400 returns 0. That pair is the null value. Otherwise the count word.
STACKPUSH	154537	Switches on param_2 >> 10, the type field above the 10-bit index.
STACKPUSHONE	154643	If the current chunk's used count equals its capacity, FUN_1368_0de0 allocates another. Then VALUECOPY. A count of 0xFFFF is not incremented.
STACKPOP	154759	Empty current chunk: CDBSETPLERR(..., 0x1FA2, 2, ...). Otherwise decrement, copy the slot, call FUN_1368_1586. An emptied chunk steps back to the previous one.
STACKSLIP	154806	Same 0x1FA2 on an empty chunk. Removes the front element and slides the rest down. Also calls FUN_1368_1586.
STACKELEMENT	154343	Index from the end: count - index - 1, via FUN_1368_0d94. Out of range is CDBSETPLERR(..., 0x38, 2, ...). An empty slot comes back as type 1 / subtype 0x400.
STACKSETELEMENT	154486	Same end-relative index, then VALUECOPY.
STACKFREE	154435	VALUEFREE each slot, TBHEAPFREE each chunk, then the header.
FUN_1368_0ab8	154705	Push, rotate the new elements to the front, and STACKPOP back to the old size on failure.
FUN_1368_01d2	154205	A count word of 0xFFFF is CDBSETPLERR(..., 0x1FFF, 2, ...).

STACKPUSH cases 1, 2, 5, 7, 10, 0xB, 0x14, 0x15, 0x16 go straight to STACKPUSHONE. Cases 8 and 0xC go through VALUEVALUE, then push with the low field | 0x800. Case 0xD pushes three words from the pointed-to object, then one more. Case 0x10 is a stack: push each element. The default converts with VALUETOSTRING and calls FUN_1368_0236 (154232), which splits the string on commas through CDBCHUNKLAST and CDBCHUNKFIND.

FUN_1368_1586 (155393) decrements the count word unless it is already 0xFFFF, in which case it calls FUN_1368_1538. The 12 October map puts the name STACKSLIP at 1368:1586. This decompile has both that address-shaped function and a separate export named STACKSLIP. The map is a different analysis, as chapter 11 says about other offsets. Behavior above uses the decompile names. A live break uses the WDW name and the WDW offset.

The TBX wrappers that were read are one call and a return: TBXSTACKPOP (115944), TBXSTACKFREE (115955), TBXSTACKPUSH (116277), TBXVALUENEWSTACK (116288).

One boot, DGROUP 347F, from the 17:38 batch. Same selector, one segment. The breakpass marked the whole STACK* set as skip. The skip was right about the reader. It was not a description of the functions.

Export	That boot
VALUENEWSTACK	4537:0000
STACKELEMENT	4537:0444
STACKSETELEMENT	4537:06B0
STACKGETSIZE	4537:0738
STACKPUSH	4537:0778
STACKPUSHONE	4537:09A2

19.5 The resource manager

RESMGR* is the book-resource manager. It is not the NE loader and not the reader.

RESMGRCONNECT (20043). CDBINIT. LOCALALLOC of 0xE bytes, seven words. Two MAKEPROCINSTANCE thunks. CDBCREATEINSTANCE(3, ...) takes the first thunk. HEAPREGISTER takes the second. The session words are: [0] instance, [1]/[2] heap far pointer, [3]/[4] and [5]/[6] the two proc-instances. Failure bytes: 0x0F if the local alloc fails, 0x0D if HEAPREGISTER returns null, 0x39 ('9') if a proc-instance fails. The failure path undoes whatever was created. RESMGRDISCONNECT (20144) destroys the instance, deregisters the heap, frees both thunks, and LOCALFREEs the session.

RESMGROPENBOOK (20177) calls CDBCREATEBOOKREF. RESMGRCLOSEBOOK (20197) calls FUN_1048_102e and CDBCLEARBOOKREFS.

RESMGRSENDMSG (20219) switches on the word at message +10.

Case	Call
1	FUN_1028_12d4, then FUN_1030_06a8
2	RESMGREXTRACT
3	RESMGRREPLACE
4	RESMGRENUM
5	Sets a flag, then RESMGRSENDMESSAGE
6	RESMGRGETNAME
7	RESMGRSETNAME
default	Stores 1 at message +0xC

RESMGRSENDMESSAGE is a separate export (line 21836). On the restart boot it was 3817:1C82, enter 0012. Its body was not walked here.

RESMGREXTRACT (21513) is FUN_1028_23ec(0, ...). RESMGREXTRACTGDI (21523) is FUN_1028_23ec(1, ...). The 12 October map puts the name RESMGREXTRACTGDI on 1028:23EC, which is the address form of FUN_1028_23ec, the shared body. The WDW stop to keep is the name plus the WDW offset: RESMGREXTRACT at 4D27:1710 on the first boot. The keepers table has RESMGREXTRACTGDI at 4D27:1730 on that same selector. FUN_1028_23ec itself was not read in this pass.

RESMGRENUM (22381) takes a code *. Each step calls FUN_1008_0422, then (*param_4)(...). The loop stops when the callback returns 0, or when the status byte is nonzero. A status of 'B' is cleared before the stop. The return is the number of steps. The 12 October map names 1008:0422 RESMGRGETREFCNT. In this decompile that address is the helper the enum loop calls. The export RESMGRGETREFCNT is a different function. Do not merge them.

Static callers of RESMGRENUM: RESMGRSENDMSG case 4, FUN_10b0_52e8 (47305), FUN_1300_029e (139318 and 139350), FUN_1308_0c2e (140211, the decompile has the string ACCELERATOR at that call). The callback arguments at 47305 and 140211 are small integers in the listing (0x5358, 0xDEC). Those are not named functions in this pass.

RESMGRREPLACE (20829) builds a message whose first word is 0x1A and whose size word is 6, then calls RESMGRSENDMESSAGE. Types 0x13 and 0x10 copy two extra words into that message. FUN_1028_0d52 calls

RESMGRGETREFCNT only when the type word is 0x13.

RESMGRGETREFCNT (22579) calls FUN_1028_2e76, then FUN_1008_0438, then OBJUNLOCK. FUN_1008_0438 is the address the map labels CDROMOBJINFO_OHANDLECHECK. The label is the map's. The body of 0438 was not read here.

Two more indirect calls sit in this segment. FUN_1028_2680 (22469), when a type id is < 0x14, calls the code pointer at that type's table entry. FUN_1028_2770 (22535) walks 0x14 entries from DAT_1630_0154 and calls the code pointer at +4. The map names 1028:2770 RESMGROPENBOOK. This decompile's export of that name is the CDBCREATEBOOKREF function at line 20177. The loop is not the open-book export.

VALUENEWRESOURCE is defined once, at line 90660. Nothing else in the C file calls it by that name. The callers are the WDW Calls window.

19.6 What the breakpass actually hit

Startup order on the first 2026-10-15 boot, heap and resource rows only. The full table is maps/wdw-to-ghidra.md. TBHEAPCLEARCRITSECT was not the last stop before the window. VALUEFREE was, two stops later.

Order	That boot	Export
6	450F:0E3E	TBHEAPSAVECRITSECT
9	450F:0E70	TBHEAPRESTORECRITSECT
13	450F:0840	TBHEAPALLOC
14	450F:0EE2	HEAPPOINTERTOVALUE
16	4D27:1710	RESMGREXTRACT
24	450F:0EC2	TBHEAPCLEARCRITSECT
26	4457:0B38	VALUEFREE, before the main window

Later the same day, still DGROUP 347F: HEAPCREATEFRAME 450F:0350, HEAPGETHANDLER 450F:0FEE, HEAPRELEASEFRAME 450F:05D6, RESMGRENUM 4D27:2522. HEAPRELEASEFRAME was not read in this pass. The batch note said skip HEAP* and RESMGRENUM. That is still the right call for finding the reader.

The restart (DGROUP 2977) hit the same offsets on a new selector. HEAPREGISTER 343F:0000, enter 0014, compares the word at 11AE (DAT_1630_11ae). TBHEAPALLOC 343F:0840, enter 0014, add ax,4, which is the header. HEAPPOINTERTOVALUE 343F:0EE2.

VALUENEWRESOURCE (4267:0EFC, enter 000E) and RESMGRGETREFCNT (4D27:2838, enter 0008) showed the same unnamed return addresses: 4D87:0102, 0ED7:1E5B, 4C0F:1C72, 518F:4776, 322F:180D. The Calls note for VALUENEWRESOURCE also prints 4D87:01C72 and a second 0ED7:1E5B. 4D87:01C72 is one digit away from 4C0F:1C72 and does not repeat on RESMGRGETREFCNT. Treat it as a bad copy of that line, not as a sixth caller, until a screenshot says otherwise. RESMGRREPLACE is an additional named caller of RESMGRGETREFCNT, which matches FUN_1028_0d52.

322F is the unnamed segment already on CDBFILEOPEN (322F:3252) and CDBCHUNKFIND (322F:0577, 322F:0E1E). 4D87:00ED is a caller of VALUEARRAYCHKDIMS. Same selector as 4D87:0102, different offset. Do not give any of these a name.

19.7 What this does not name

The reader is still the unnamed walk after CDBFILEOPEN and before the first paint. Chapter 4. The next break is still CDBCREATEPAGEREF or CDBSETVALUEEX.

Not named here: the body of the HEAPREGISTER proc-instance, the type-0x22 callee at object +0x2C, FUN_1028_23ec, FUN_11f0_2440, HEAPRELEASEFRAME, and the five 322F / 4D87 / 0ED7 / 4C0F / 518F return addresses.

CDBSENDCREATE is still author-only and was not on these Calls stacks.

20. CDB and OBJ

Date: 2026-10-16

Status: The split between the CDB* and OBJ* exports of TB80BAS. Counts are the NE export table of the unpacked DLL (606 named entries). Call sites are TB80BAS.DLL.c. Relocation counts are the unpacked images. This does not name the record reader.

Claims

Id	Grade	Statement
S-001	Known	Ordinals 1 through 56 are CDB*, with no other name in between. Ordinal 57 is LOCKPATTERN. There are 128 CDB* exports in all, across 26 NE segments.
S-002	Known	There are 29 OBJ* exports. Twenty-six of them are NE segment 47 (Ghidra 1170). OBJARENACHECK is segment 44, OBJEXPORTMSGNEAR is 49, OBJINIT is 103. OBJECTNAME (ordinal 160, segment 61) is not one of them.
S-003	Not true	"CDB* is the header and OBJ* is the definition of the same functions." Both are exported functions with bodies. Three stems occur on both sides (INIT, FILEOPEN, FILECLOSE). Those are not the same function.
S-004	Known	CDBGETVALUE, CDBSETVALUE, and CDBSENDVALUE are one call each, into the matching *EX export, with 0 in the extra argument. CDBUNREGISTEROLE2WINDOW is CDBREGISTEROLE2WINDOW(0, 0, ...).
S-005	Known	CDBGETVALUEEX locks with OBJLOCKNEAR, calls the code pointer at the 0x76 slot, then OBJUNLOCK. The same three calls (lock near, the operation, unlock) are in CDBSETVALUEEX and CDBSENDVALUEEX.
S-006	Known	CDBFILEOPEN does not name OBJFILEOPEN. The decompile reaches it through FUN_1048_1cc4 and through FUN_1030_02fc. Those two call FUN_1048_1766 and FUN_1048_2174, and those call OBJFILEOPEN.
S-007	Known	Outside BAS, the modules whose import table names TB80BAS are RTM, CMP, NET, and FLT. RTM's BAS relocations are CDB*, VALUE*, STACK*, HEAP* / TBHEAP*, and RESMGR*. The only OBJ* name among them is OBJBOOKVERSION.
S-008	Thought	CDB* is the book API the other modules link. OBJ* is the object file and the object memory inside BAS. The DLL never expands the letters. The C++ objects are the mangled TBX* classes, which are a third layer.

20.1 Not a header and a C file

A header would declare a function whose body lived in one object file. That is not this split.

The DLL contains no .c or .h names. Copyright string: Asymetrix, 1990–1995. Both prefixes are in the resident and nonresident name tables, so both are linkable. CDBGETVALUE is one call, and the function it calls is CDBGETVALUEEX, not an OBJ* function. The short-name / EX-name pair is inside CDB*.

CDBINIT (line 30304) is 193 lines of device caps, pens, and colors. At line 30465 it calls OBJINIT(0x242, 7, param_1) and returns if the error byte was set. OBJINIT (line 142908, NE segment 103) runs once: a profile int via GETTBKINIPROFILEINT, then FUN_11a8_0000. The book init starts the object layer. It is not a prototype for it.

The three shared stems:

Stem	Book side	Object side	What the decompile shows
INIT	CDBINIT, ordinal 1	OBJINIT, ordinal 225	One call, above.
FILEOPEN	CDBFILEOPEN, ordinal 8, line 27592	OBJFILEOPEN, ordinal 227, line 76162	Not a direct call. Section 20.3.
FILECLOSE	CDBFILECLOSE, ordinal 11	OBJFILECLOSE, ordinal 246	CDBFILECLOSE does not call it. CDBFILESAVEAS does, once, and also calls OBJFILECOPY.

CDBFILENEW calls OBJLOCK, OBJTOUCH, and OBJUNLOCK. CDBSETFOCUS calls OBJPRINCIPALOO. CDBDELETEOBJECTVALUE calls OBJLOCK, OBJLOCKNEAR, OBJFREENEAR, and OBJUNLOCK. No OBJ* body calls a CDB* function. OBJECTNAME does: CDBVALUEBOOKTOSTRING and CDBSETPLERR. That export is not in segment 47.

The 2026-10-12 symbol map line `11A8:04F6 OBJFILEOPEN` is `FUN_11a8_04f6`. The decompile calls that `FUN_` by name a few lines above the real `OBJFILEOPEN`. The export offset is `177C`. Chapter 11 already has that offset. Do not use the map line.

20.2 What each side is

Ordinals 1 through 56 are the book surface, in order: instance and thread, file, view, paint, the ref constructors (`CDBCREATEBOOKREF`, `CDBCREATEPAGEREF`, `CDBCREATEBKGNDREF`, `CDBCREATEGRAPHICREF`), the value bus, `CDBSENDCREATE` (ordinal 51, still author-only, chapter 4), `CDBNEWOBJECT`, then the four text chunkers. Fonts and patterns start at 57. Later `CDB*` exports exist (`CDBSETPLERR`, the OLE forwards, the break addresses). They are not in that first run.

`OBJ*` in segment 47 is the object machinery:

Group	Exports
Allocate and free	`OBJNEWNEAR`, `OBJNEWNEAREX`, `OBJREALLOCNEAR`, `OBJREALLOCNEAREX`, `OBJFREENEAR`, `OBJFREENEAREX`, `OBJFREE`, `OBJNEWSEG`, `OBJFREESEG`, `OBJTOUCH`
Lock	`OBJLOCK`, `OBJUNLOCK`, `OBJLOCKSEG`, `OBJLOCKNEAR`
File	`OBJGETFILE`, `OBJGETHDRLONG`, `OBJFILEOPEN`, `OBJFILECLOSE`, `OBJFILECOPY`, `OBJFILETYPE`, `OBJREADFAR`, `OBJBOOKVERSION`
Handle and principal	`OBJOOTOOH`, `OBJOPTOOH`, `OBJPRINCIPALOO`, `OBJPRINCIPALOP`

`OBJNEWNEAR` (line 75226) is a body, not a macro: `FUN_1258_001c`, then `FUN_1178_0518`. About seventy functions in BAS call it. None of those callers is a `CDB*` export. The book exports lock and free. The unnamed functions allocate.

`CDBGETVALUEEX` (line 38488) is the pattern the value bus uses. The index expression at line 38671 is `(param_7 & 0xff) * 0x76` added to the word at `local_6 + 2`. The code pointer is the dword at `+4` of that slot. `OBJLOCKNEAR` runs, the pointer is called, then `OBJUNLOCK` (line 38690). When the other handle is set, the call is `FUN_1008_03f6` instead, still between the same lock and unlock. `CDBSETVALUEEX` and `CDBSENDVALUEEX` do the same lock. The stride is the live object from chapter 4. This is not a second measurement of the size, and it is not `TBHEAPALLOC`.

`CDROM*` and `CDROMOBJINFO_*` are a different prefix (self-test, drop target, object-info procs). They are not this `OBJ*` segment. `VALUEFROMCDBVALUE` is the converter RTM imports (17 relocation sites). A CDB value and a script `VALUE*` are not the same word. `HEAPFINDCDBVALUE` and the `0xCDBF` argument stay chapter 19. Do not read that tag as one of these exports.

20.3 File open

`CDBFILEOPEN` calls `FUN_1030_02fc`, `FUN_1030_0d08`, `FUN_1048_1cc4`, and `FUN_1048_0b46`. It does not mention `OBJ*`.

`FUN_1048_1cc4` (line 29115) is also called by `CDBFILEMERGE` and `CDBCREATEBOOKREF`. Its decompile calls `FUN_1048_2174` and `FUN_1048_1766`. Both of those call `OBJFILEOPEN`. `FUN_1030_02fc`, which `CDBFILEOPEN` also calls, calls `FUN_1048_1766` as well. `FUN_1048_1766` has other callers (`FUN_1028_3adc`, `FUN_1040_0ca8`, `FUN_1048_159a`, several `FUN_1078_*`, `FUN_10c8_144c`). It is a shared file helper, not the open path alone.

Two more callers of `OBJFILEOPEN` sit outside that segment: `FUN_1170_34cc` (inside segment 47) and `FUN_1350_1000`. The bodies of `1cc4`, `1766`, `2174`, `34cc`, and `1000` were not read. The claim is the call graph, not a description of what each one parses.

Chapter 4 stands. `OBJFILEOPEN` is the large file frame (`enter 0x25A` on the boot that stopped there). A 4-byte read on that path was an internal pointer, not a page record. Reaching it from `CDBFILEOPEN` does not make it the reader.

20.4 Who links which

Relocation sites against TB80BAS in the unpacked images:

Module	Sites	What they are
TB80RTM.EXE	983	373 CDB* (98 distinct names), plus VALUE*, STACK*, HEAP* / TBHEAP*, RESMGR*. OBJECTNAME is 9 sites. OBJBOOKVERSION is 6. No OBJLOCK, OBJNEWNEAR, OBJFILEOPEN, or OBJUNLOCK.
TB80CMP.DLL	44	CDBLOOKUPNUMBER (2), CDBSETPLERR, CDBGETVALUE. The rest are VALUE* and HEAP* / TBHEAP*.
TB80NET.EXE	1	CDBNETCOMINIT.
TB80FLT.DLL	1	CONVERTTORUNTIMEMEDIAPATH. Not CDB* or OBJ*.

TB80W16, TB80OCX, TB80VBX, TB80UTL, TB80OLE, TB80MAIN, and TB80RUN do not name TB80BAS in the module-reference table. The packed originals are the wrong file for this count: every segment but the stub has the relocation flag clear. The unpacked images are the ones with 134 relocating segments in RTM.

One hundred of the 128 CDB* names appear in some other module's relocation table. The object manager does not.

20.5 The third layer

TBX* is 117 exports, starting at ordinal 291 (TBX32MSGTHUNK). Some names are the other two prefixes with TBX glued on: TBXCDBGETVALUE, TBXCDBSENDVALUE, TBXCDBSETPLERR, TBXOBJPRINCIPALOO, TBXOBJSENDNEAR, TBXCURRENTCDBID, TBXENUMCDBIDINSTANCES. Those are still inside BAS. They are not a second DLL.

The C++ objects are the mangled exports: TBXBASE, TBXUSER, TBXLAYER, TBXOBJECT, TBXBOBJ, TBX32OBJECT, OCXSTUBOBJECT. Each publishes SENDPROC, GETPROC, and SETPROC. CDROPTARGET is the drop target. CVARIANTVALUEUTILITIES is the variant bridge. That is the C++ object layer. OBJ* is __stdcall16far C, not those methods.

GETCDBID and TBXCURRENTCDBID are exports. The binary does not say what the three letters stand for. Do not expand them.

20.6 What this does not change

The reader is still the unnamed walk after CDBFILEOPEN and before the first paint. Chapter 4. The next break is still CDBCREATEPAGEREF or CDBSETVALUEEX.

Not read for this chapter: the bodies of FUN_1048_1cc4, FUN_1048_1766, FUN_1048_2174, FUN_1170_34cc, and FUN_1350_1000; the principal-op callee; every CDB* export after the value bus. CDBSENDCREATE was not re-opened.

21. What the stress run broke

Date: 2026-10-16

Status: The Python in openscript_analysis and the Foolbook engine, run under a stress suite. This is not a timing of TB80BAS. It does not name the record reader.

Claims

Id	Grade	Statement	Source
P-001	Known	Four nested step loops from 1 to 100 do not finish. Each loop had its own cap of 10,000, so the caps multiplied.	engine.py before the shared budget
P-002	Known	Eighty nested if lines raised RecursionError.	The same runners

Id	Grade	Statement	Source
P-003	Known	`name = "a and b"` was split on the `and` inside the quotes, so the comparison was false.	`_eval`
P-004	Known	`ScriptHeap.free` keyed only on the 10-bit index. After the index was reused, freeing the old value freed the new block.	`heap.py`
P-005	Known	`GHeap.alloc` did not look at `data + zero + 0x10`. A request larger than the block was accepted.	`GHeap.alloc`
P-006	Known	One `0xFF` opcode appends up to 256 bytes after the output cap had already been checked.	`expand.decompress`, Foolbook `decode`
P-007	Known	A resource row whose `type` was the word `Bitmap`, or a row that was not a dict, aborted `adopt` before the later rows.	`resmgr.adopt`

21.1 What runs now

The runners share one budget of 200,000 line steps and stop nesting at 48. The four-deep step returns in the stress suite, which itself finishes in under two seconds, and the action list contains `stopped`. The deep `if` does the same. It does not walk off the Python stack.

`and`, `or`, and the comparisons are scanned outside quotes. `&&` in the TBX runner is the same. `if name = "a and b"` now stores `1`.

`free` and `pointer` require the offset as well as the index. A stale value does not resolve, and it does not free the block that reused the index. `GHeap` refuses a request that would pass the end of its block, and a second free of the same ident returns false.

`decompress` and Foolbook `decode` clip each opcode to the room that is left. A 4,000-opcode `0xFF` run with `max_out` of 1,000 stays at 1,000 characters.

A bad resource row is skipped. The row with `type` 1 is still adopted.

A push of 3,000 comma-separated pieces still stops at the index limit (`0x400`) with `0x1F7A`. That path was already right. It stays under a second. The index limit was not raised. Raising it would not match the 10-bit value.

21.2 What was not made faster

The heap's per-frame block list is still a list. At 1,024 indexes a free is not the cost. The cost was the nested step, which did not return.

`tbpipeline` `layer_of` is a prefix test. It was not on this path.

The suite after the change is `openscript_analysis.test_stress`, `openscript_analysis.test_machine`, `engine.test_stress`, and `engine.test_runtime`. 38 tests, all passing. None of them reads a page record.

22. Value, VBX, and EXECSCRIPT

These are three mechanisms. A name belongs to one of them, or to none. It does not belong to two.

A VBX is a Visual Basic extension control. It is not the `0x76` page object. The value bus is not a VBX. `EXECSCRIPT` is not a VBX. The page object is chapter 4. The heap tags are chapter 19. The book API that locks the page object is chapter 20. This chapter is the split between the three names that get confused with that object.

Claims

Id	Grade	Statement	Source
V-001	Known	A script value is a tag and a payload. `VALUENEWLONG` is tag `0xB`, four bytes. `VALUENEWDOUBLE` is `0xA`, eight. `VALUENEWSTACK` is `0x10`, a `0xC`-byte header.	Chapter 19, the `TBHEAPALLOC` sites
V-002	Known	`CDBGETVALUE`, `CDBSETVALUE`, and `CDBSENDVALUE` are one call into the `EX` form. The `EX` form locks the `0x76` object, calls the code pointer in the slot, and unlocks. The value is what is passed. The object is what is locked.	Chapter 20
V-003	Known	`VBXINIT` loads `TB80VBX.DLL` and calls `GetProcAddress` for `VbxInit`. It does not allocate the view.	Appendix K, the body that was read
V-004	Known	`VBXNEWVIEW` pushes `0x11C` and calls far. The view magic in that reading is `0xABC1`. The size is not `0x76`.	`maps/wdw-to-ghidra.md`; Appendix K
V-005	Known	`TBXEXECSCRIPT` is BAS ordinal 405 at `1268:060c`, decompile line 115148. The other five names in the family are separate exports.	BAS export map
V-006	Thought	`EXECSCRIPT` is the TBX edge that runs a script. The bodies were not read. The stack thunks that were read are one call and a return. That shape is not claimed for this family.	The names, and chapter 19 on the thunks that were read
V-007	Not true	A VBX view, a script value, and `EXECSCRIPT` are the page object, or are each other.	This chapter

22.1 Value

The constructors whose tags were read:

Export	Tag	Size
`VALUENEW`	`0`	The caller's
`VALUENEWSTRING`	`9`	`param_1 + 3`
`VALUENEWDOUBLE`	`0xA`	8
`VALUENEWLONG`	`0xB`	4
`VALUENEWPOINT`	`0xC`	4
`VALUENEWRECT`	`0xD`	8
`VALUENEWDATE`	`0xE`	7
`VALUENEWTIME`	`0xF`	3
`VALUENEWSTACK`	`0x10`	`0xC`
`VALUENEWCSTR`	`0x11`	In the helper, not in the export's own body
`VALUENEWCOLORTABLE`	`0x12`	`0x100`
`VALUENEWICON`	`0x13`	`param_1 + 2`

`VALUENEWBOOK`, `VALUENEWRESOURCE`, and `VALUENEWGRAPHIC` are constructors in segment `4267`. The `enter` sizes on the one-boot capture are frame sizes. They are not the tags in the table. A frame of `0xE` is not tag `0xE`, and tag `0xE` is a date.

`TBXVALUENEWSTACK` is a thunk onto the value constructor. It is still the value mechanism. The `TBX` stem does not make it `EXECSCRIPT`.

The bus is `CDBGETVALUE`, `CDBSETVALUE`, `CDBSENDVALUE`, the `EX` forms, and `CDBPARENTVALUE`, `CDBDEREFVALUE`, `CDBLOCKVALUE`. They use a value. They are not a value allocation, and they are not a VBX.

22.2 VBX

A VBX is a Visual Basic extension control. The host module is `TB80VBX.DLL`. A file that ends in `.vbx` is the control that host loads. BAS does not implement the control. BAS reaches the host.

Export	What the body is	What it is not
VBXINIT	Loads TB80VBX.DLL. GetProcAddress of VbxInit.	Not the allocator. Not a view.
VBXNEWVIEW	The view. 0x11C bytes. Magic 0xABC1.	Not 0x76. Not a script value.
CDBREGISTERVBXFORM	Registers a form. Seen on open. Not the reader.	Not VBXNEWVIEW.

TB80VBX does not name TB80BAS in the module-reference table. The call goes the other way: BAS loads the DLL. Opening TB80VBX.DLL is a different image. The decompiler will not follow GetProcAddress into it.

22.3 EXECSCRIPT

Export	Line	Address, when the map has one
TBXEXECSCRIPT	115148	Ordinal 405, 1268:060C
TBXEXECSCRIPT_CALLBACK	115128	1268:0488
TBXEXECSCRIPTDYNAMIC	115215	
TBXEXECSCRIPTVARIANT	115231	1268:0516
TBXAUTOEXECSCRIPTVARIANT	115293	12F8:0018
TBXEXECSCRIPTVARIANTSPCL	115358	1008:09F8

These are six exports. They are not one function with six names. The body grade is Thought, and only for the role: the TBX edge that runs a script. The operation is not Known until a body is read. Do not fill the gap with the stack thunks. Do not file the family under VBX because both words sit near "control" in a search.

22.4 What the suite does with the split

The classifier lives in openscript_analysis only. Foolversity does not keep a second copy. CDBGETVALUE and CDBREGISTERVBXFORM are the book API, in cdb.py. OBJFILEOPEN and OBJNEWNEAR are the object file, in obj.py. Those two are not the same record: the book API locks an object and does not allocate it, and the object layer is the 0x76 array. VALUENEWLONG stays a heap constructor. VBXNEWVIEW stays the control view. TBXEXECSCRIPT stays the script edge. A value constructor refuses a size of 0x76.